# From CPU SDF Shapes to GPU-Resident Surface Nets Meshing

A Detailed Account of Adaptive Octree Construction,

GPU Topology Extraction, and Real-Time Rendering

Vulkan Engine Technical Report

Henrique Rocha

October 26, 2026

**Abstract**

This report describes, end to end, how the engine turns an implicit signed-distance-model (SDF) scene into a rendered, crack-free, adaptive triangle mesh. The construction is deliberately split between two processors. The CPU owns the *semantic* structure: it evaluates the model's distance field, builds and edits an adaptive octree of corner samples, chooses the level-of-detail (LoD) ladder, and publishes chunk metadata. The GPU owns every *Cartesian* operation: it flattens a chunk's octree neighbourhood into compact tables, walks the twelve cube edges, stitches LoD discontinuities, solves one dual vertex per surface cell, emits the triangle stream, and writes the final 64-byte vertex and index payloads directly into the renderer's device-local packed pools. No runtime vertex is read back to the host.

The document is organised as a sequence of algorithms with the data structures and invariants that make them correct. It covers the SDF and CSG foundations, octree storage and threading, the CPU shape recursion, simplification and the chunk ladder, the CPU reference walker used by the offline exporter, the exact GPU port in `OctreeTriangles.comp`, the single-dispatch host orchestration, packed-pool span binding and alignment, GPU culling with a hole-free hierarchical LoD gate, shadows and ray tracing, GPU-side vegetation and lava effects, and the synchronization and validation rules that keep the pipeline clean. It closes with measured costs, memory budgets, failure modes, and a set of correctness invariants that a future change must preserve.

**Keywords:** signed distance functions; adaptive octree; Surface Nets; dual contouring; GPU meshing; compute shaders; level of detail; crack-free stitching; Vulkan; indirect rendering.

## Contents

**Notation and conventions** **9**

**1 Introduction** **9**

1.1 The problem . . . 9

1.2 Why not mesh on the CPU? . . . 9

1.3 Contributions . . . 10

1.4 Reading guide . . . 11

**2 Foundations and related work** **11**

2.1 Implicit surfaces and signed distance fields . . . 11

2.2 CSG operations . . . 11
2.3 Dual contouring and Surface Nets . . . 12
2.4 Adaptive octrees and LoD . . . 12
2.5 Why two processors? . . . 12
2.6 Position in the literature . . . 13

**3 The SDF model and its discretisation 13**
3.1 Primitives and transformations . . . 13
3.2 Corner sampling . . . 13
3.3 Classification . . . 14
3.4 Trilinear reconstruction and its gradient . . . 14
3.5 The trilinear lerp with sentinels . . . 15
3.6 The QEF vertex . . . 15
3.7 Brushes, painters, and mapping state . . . 15

**4 The CPU octree data structure 16**
4.1 Node layout . . . 16
4.2 Cube corners and edges . . . 16
4.3 Types and sentinels . . . 17
4.4 Level of detail and chunk ladder . . . 18
4.5 Memory model . . . 18
4.6 Concurrency . . . 18

**5 CPU shape construction: `Octree::apply` 19**
5.1 Entry point and root expansion . . . 19
5.2 The recursive `shape` call . . . 19
5.3 Sampling and caching . . . 19
5.4 The three early-out rules . . . 19
5.5 Result composition . . . 21
5.6 Painting . . . 21
5.7 Node publishing and versions . . . 21
5.8 Threading . . . 22
5.9 Why the CPU never solves a vertex . . . 22

**6 Simplification and the LoD ladder 22**
6.1 What simplification means here . . . 22
6.2 The error criterion . . . 23
6.3 The material and chain guards . . . 23
6.4 The chunk-border guard . . . 23
6.5 Level propagation . . . 23
6.6 Why every rung is published . . . 24
6.7 Chunk versions and rebuilds . . . 24
6.8 Cost model . . . 24

**7 The CPU reference walker** **24**
7.1 Frontier and scan cells . . . 25
7.2 Twelve edges and edge spans . . . 25
7.3 Why one edge can produce many segments . . . 25
7.4 Collecting breaks . . . 25
7.5 The break list and the interval stack . . . 26
7.6 Owner selection . . . 27
7.7 Polygon reconstruction . . . 27
7.8 Winding . . . 27
7.9 Triangle fan and duplicate rejection . . . 27
7.10 The CPU triangle handler . . . 28
7.11 Why keep the CPU walker? . . . 28

**8 GPU-resident meshing: design constraints** **28**
8.1 The contract . . . 28
8.2 Why one geometry pass? . . . 28
8.3 Overflow without out-of-bounds writes . . . 29
8.4 Determinism . . . 29
8.5 Precision strategy . . . 30

**9 Flattening the octree neighbourhood** **30**
9.1 What the shader needs . . . 30
9.2 Root-consistent ancestor chain . . . 30
9.3 Descending the table . . . 31
9.4 The scan-cell list . . . 31
9.5 The descent anchor . . . 32
9.6 Why the margin is one cell . . . 32
9.7 Vertex slots . . . 32
9.8 Table order and determinism . . . 32
9.9 Failure modes . . . 32

**10 The GPU geometry pass** **33**
10.1 Invocation model . . . 33
10.2 Node accessors . . . 33
10.3 `findCellAt` . . . 33
10.4 Edge span construction . . . 34
10.5 The ulp nudge and the zero guard . . . 34
10.6 The scan loop . . . 34
10.7 Diagnostics . . . 34

**11 Step-by-step adaptive stitching** **35**
11.1 The exact problem . . . 35
11.2 Why a fixed stitching table is not used . . . 35

11.3 The recursion, formally . . . 35
11.4 The GPU stack . . . 36
11.5 The sample offset . . . 36
11.6 Break capacities and overflow . . . 36
11.7 The global break list . . . 36
11.8 Owner and quadrant sampling . . . 37
11.9 Consequences . . . 37

**12 Polygon reconstruction, winding, and triangle emission 37**
12.1 From four quadrants to a ring . . . 37
12.2 Rotating the owner to the front . . . 37
12.3 Three-ring and four-ring fans . . . 38
12.4 The winding oracle . . . 38
12.5 Degenerate triangles . . . 38
12.6 Direct index append and lazy vertices . . . 39
12.7 Triangle order and visual invariance . . . 39
12.8 The boundary quad . . . 39

**13 Step-by-step QEF vertex solve and payload 40**
13.1 Why a QEF instead of an average . . . 40
13.2 The twelve crossings . . . 40
13.3 Accumulation . . . 40
13.4 Corner snap . . . 40
13.5 Sentinel handling . . . 41
13.6 Clamping . . . 41
13.7 Memoization . . . 41
13.8 The payload . . . 41
13.9 One dense slot per node . . . 41

**14 The geometry pass: indices and lazy vertices 42**
14.1 One dispatch, two outputs . . . 42
14.2 Index values . . . 42
14.3 Vertex reconstruction . . . 42
14.4 The index base and alignment . . . 43
14.5 The vertex base and alignment . . . 43
14.6 Overflow accounting and growth . . . 43
14.7 Empty chunks . . . 43

**15 Host orchestration: one submission per chunk 44**
15.1 The driver . . . 44
15.2 The input upload . . . 44
15.3 Capacity management . . . 44
15.4 Capacity plan and the provider callback . . . 44

15.5 Overflow growth . . . 45
15.6 Publication . . . 45
15.7 Why the host reads only counters . . . 45
15.8 Error policy . . . 46

**16 Synchronization, queues, and lifetime** **46**
16.1 Queues in use . . . 46
16.2 The barriers inside the topology submission . . . 46
16.3 The fence and the deferred release . . . 47
16.4 Consolidation . . . 47
16.5 The queue-submission mutex . . . 47
16.6 Deferred destruction order . . . 48
16.7 Why no host-side mesh wait . . . 48

**17 From pool span to rendered frame** **48**
17.1 Stable draw entries . . . 48
17.2 The draw command . . . 49
17.3 Depth and offscreen composition . . . 49
17.4 Shadows . . . 49
17.5 Ray tracing . . . 49
17.6 Material and mapping . . . 50
17.7 The frame graph . . . 50

**18 GPU culling and the hole-free LoD gate** **50**
18.1 One dispatch, many streams . . . 50
18.2 Frustum test . . . 50
18.3 The hierarchical rung gate . . . 50
18.4 The visible-LoD stamp . . . 51
18.5 Vegetation follows the chunk . . . 51
18.6 Water pin . . . 52
18.7 Indirect count . . . 52

**19 GPU-resident effects: grass and fire** **52**
19.1 The shared span . . . 52
19.2 Vegetation placement . . . 52
19.3 Why not one instance per triangle? . . . 52
19.4 The vegetation draw . . . 53
19.5 Lava/fire anchors . . . 53
19.6 Ownership and cleanup . . . 54

**20 Performance and memory** **54**
20.1 Cost model . . . 54
20.2 Measured topology rates . . . 54

20.3 Throughput . . . 54
20.4 Per-chunk utilization profiling . . . 55
20.5 Memory budget . . . 55
20.6 Streaming behaviour . . . 56
20.7 Failure costs . . . 56
20.8 Interpretation . . . 56

**21 Correctness invariants and validation 56**
21.1 The invariants . . . 56
21.2 Validation layers . . . 57
21.3 GPU-assisted validation . . . 57
21.4 Self-check against the CPU walker . . . 57
21.5 Determinism tests . . . 57
21.6 Synchronization validation . . . 58
21.7 The queue-threading bug . . . 58
21.8 Regression checklist . . . 58

**22 Discussion, limitations, and future work 58**
22.1 What the split buys . . . 58
22.2 The remaining host cost . . . 59
22.3 The mesh is not the only consumer . . . 59
22.4 Adaptive LoD and cracks . . . 59
22.5 The coarse-rung density cap . . . 59
22.6 The fire anchors . . . 59
22.7 What would break the design . . . 60
22.8 Future work . . . 60

**23 Conclusion 60**

**A Storage layouts and constants 61**
A.1 Octree node record (host) . . . 61
A.2 GPU node table (flattened) . . . 61
A.3 Packed vertex record . . . 61
A.4 Push constants . . . 62
A.5 Constants . . . 62

**B The complete scan-cell algorithm 63**

**C Glossary 63**

# List of Figures

1 End-to-end pipeline. Blue nodes execute on the CPU; orange nodes execute on the GPU. The only data crossing from GPU to CPU are integer counters used for span sizing and statistics. Chunk geometry itself never returns to the host. . . . . 10
2 Two-dimensional cross-sections of the CSG rules. Blue is the solid; a subtraction removes the second field from the first; the rightmost panel illustrates a non-distance-preserving effect whose zero set and sign are still well defined. . . . . 12
3 A surface cell in two dimensions. The shaded region is the inside ($d < 0$). Sign changes on cube edges give crossing points and normals; the QEF fits a dual vertex to those planes. In three dimensions there are twelve cube edges, and the vertex is clamped inside the cube. . . . . 15
4 Corner numbering and the first four edges. The cube is drawn with $z$ receding; the corner bits are $(x, y, z)$ with bit $2 = x$, bit $1 = y$, bit $0 = z$. The GPU table stores corners in two `vec4`s and edges as twelve integer pairs. . . . . 17
5 The chunk ladder. Every surface-bearing cell whose size lies in $(\Delta/2, \Delta]$ is a chunk root at some rung. Its children are the next finer rung, and so on. The GPU gate walks this dyadic lattice from the root to choose exactly one visible rung per region. 18
6 The shape recursion. A leaf samples the new field; an unaffected interior copies the old field; an affected interior recurses, classifies the parent from its children, simplifies, paints, and (for surface chunk roots) publishes a new version. . . . . 20
7 An active edge crossing a coarse/fine cell boundary. The edge is split at every cell boundary coordinate; each sub-segment is emitted by its owner cell. Both the coarse and the fine side collect the same set of boundaries, so the resulting rings agree along the shared edge. . . . . 25
8 Left: the four quadrant cells around an edge. Right: the dual polygon is the ring of their vertices. The ring is rotated so the owner leads, then fanned. The winding is chosen from the SDF sign at the edge start, not from the normals. . . . . 27
9 The single geometry dispatch. The CPU sizes both spans from the cross-edge topology before submission; the shader appends indices directly and writes each referenced vertex payload once. A defensive overflow is reported through counters and resolved by growing only this chunk's span. . . . . 29
10 The flattened neighbourhood. The chunk subtree is walked to the target rung; the one-cell margin is included so edge samples that cross the boundary can resolve coarser neighbours. Index 0 is always the world root, because the shader's descent starts there for samples outside the chunk anchor cube. . . . . 30
11 One scan-cell invocation. Inactive edges are skipped; active edges are stitched and emitted. The entire control flow is private to the invocation. . . . . 35
12 One chunk's normal timeline. The spans are reserved from the CPU cross-edge bound before submission; the single geometry pass writes the renderer's pools directly. The host reads integers after the fence. . . . . 44
13 Packed pools and stable draw entries. Chunk spans are packed best-fit into one large vertex pool and one large index pool; the draw entry is a stable index into the indirect-command array. Rebuilding a chunk replaces only its span. . . . . 49

14 The hierarchical gate selects one rung per region. The leaf is selected near the camera; the sibling rung is selected farther away. A region can never lose all rungs because the parent/child ranges are monotone along the chain. . . . . 51
15 Two GPU-resident effects sharing the chunk span. Vegetation writes instances and copies them to an exact persistent buffer; fire resolves anchors during the build and hands them to the SDF renderer. . . . . 53
16 Observed topology cost per chunk. The floor is the cost when the worker pool is not contending with the render thread or with buffer growth. . . . . 55

## List of Tables

## List of Algorithms

1 GPU scan cell (one invocation per cell) . . . . . 63

## Notation and conventions

| Symbol | Meaning |
|---|---|
| $f(\mathbf{x})$ | signed distance field; negative inside the solid |
| $\mathcal{V}$ | an adaptive octree node/cell |
| $\mathbf{c}$ | a cube minimum corner $\mathbf{c} \in \mathbb{R}^3$ |
| $L$ | cube edge length |
| $C_i$ | cube corner $i$; bits $(x, y, z)$ with bit 2 $= x$, bit 1 $= y$, bit 0 $= z$ |
| $d_i$ | stored SDF sample at corner $i$; $+\infty$ means "no data" |
| $\mathcal{T}$ | a triangle (three corners / three dual vertices) |
| $s$ | a surface-node slot index assigned on the host |
| $\mathcal{N}$ | a flattened node table uploaded for one chunk |
| $\mathcal{P}_v, \mathcal{P}_i$ | packed vertex and index pools |
| $b_v, b_i$ | pool element base of a reserved chunk span |
| $\Delta$ | one chunk's cube edge (the chunk boundary) |
| $r$ | LoD rung index; $r = 0$ is the finest published rung |
| $\tau$ | a threshold (simplification error, break epsilon, etc.) |

The reader is assumed to know Vulkan at the level of command buffers, descriptor sets, compute dispatches, and pipeline barriers. The report does not repeat the Vulkan specification; it explains why each operation is placed where it is and which synchronization argument makes it legal.

# 1 Introduction

## 1.1 The problem

A modern terrain and solid-geometry engine must satisfy four requirements at the same time:

**R1. Authoring freedom.** Designers describe geometry with implicit primitives, Boolean operations, noise, and material painters, not with a fixed triangle list.

**R2. Adaptive detail.** Near geometry must be finely tessellated; the same region observed from a kilometre away must collapse to a coarse representation without cracks or holes.

**R3. Low host traffic.** A terrain that streams hundreds of chunks per second cannot afford a GPU→CPU→GPU round trip for every vertex.

**R4. Real-time rendering.** The result must feed frustum culling, shadow cascades, ray tracing, vegetation placement, and volumetric effects without a second geometry representation.

The engine answers with a split pipeline. The CPU evaluates the scene's signed distance field (SDF) and maintains an adaptive octree of corner samples. It never stores a triangle mesh. The GPU receives a flattened subset of that octree for one chunk, performs the full adaptive Surface Nets walk, solves one dual vertex per surface cell, and writes the resulting vertices and indices directly into the renderer's persistent packed pools. The host sees integer counters, not positions.

## 1.2 Why not mesh on the CPU?

The original engine did exactly that. `Octree::iterateTriangles` walked the tree, solved QEF vertices, and appended triangles to a `Geometry` object; a staging copy then uploaded the vertex and index arrays. The CPU path is still present as `OctreeTriangleTessellator`, but it now serves only

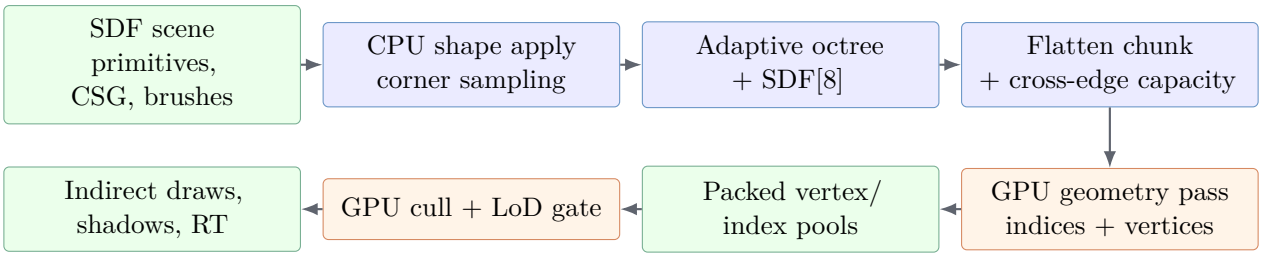



**Figure 1:** End-to-end pipeline. Blue nodes execute on the CPU; orange nodes execute on the GPU. The only data crossing from GPU to CPU are integer counters used for span sizing and statistics. Chunk geometry itself never returns to the host.

the offline Blender exporter. Moving the same walk to a compute shader produced three concrete benefits:

- **The host never touches the vertex stream.** A chunk's topology is written into a device-local span. The host reserves the span, publishes a draw command, and never reads a position.
- **The adaptive walk parallelises trivially.** One invocation owns one scan cell. All per-cell state (break lists, QEF memo, recursion stack) is private, so the only atomics are the index append, the overflow counters, and the per-node vertex-writer election.
- **The renderer and the mesher share one memory layout.** The compute pass writes the renderer's exact 64-byte `Vertex` format and chunk-local index values. There is no conversion, compaction, or intermediate mesh object.

The cost is that the GPU must see exactly the octree neighbourhood the CPU walker would have traversed: coarser neighbours for edge stitching, finer children for the descent, the world-root path, and a deterministic assignment of vertex slots. The bulk of this report is about making that flattening exact and about the single-pass GPU mesher that consumes it.

### 1.3 Contributions

This report makes the following contributions, all implemented in the current engine:

**C1.** A precise description of the CPU octree as a *corner-sample store*, including its +1-shifted LoD representation, block allocator, quadratic-error simplification, and threaded `apply` recursion.

**C2.** A root-consistent flattening algorithm that turns a chunk's octree neighbourhood into eight compact storage buffers and a scan-cell list, with host-assigned vertex slots packed into node flags.

**C3.** A single-pass GPU topology geometry stage: the CPU cross-edge count sizes both spans, and one dispatch appends the chunk-local index stream and writes each referenced vertex payload exactly once.

**C4.** Per-chunk cross-edge capacity accounting and overflow growth: an overflow is reported without out-of-bounds writes and grows only the affected chunk's span.

**C5.** An exact GPU port of the adaptive edge stitching algorithm (per-edge break collection at LoD boundaries), including its fixed capacities, overflow policy, and deterministic ordering.

**C6.** A packed-pool span protocol that binds only a chunk's slice of a large pool, respects `maxStorageBufferRange`, handles alignment padding, and publishes the draw entry only when the fence signals.

**C7.** A hole-free hierarchical LoD gate used by the raster, shadow, bounding-box, and water streams, plus the GPU-resident effect consumers (vegetation instances and lava/fire anchors) that read the same chunk metadata.

**C8.** A validation-oriented account of the synchronization, lifetime, and determinism rules that keep

the path clean across asynchronous compute, graphics, and transfer queues.

### 1.4 Reading guide

Section 2–7 cover the CPU side: SDF mathematics, octree storage, the shape recursion, simplification, and the reference walker. Section 8–14 cover the GPU meshing pass in increasing detail. Section 15–16 cover host orchestration and synchronization. Section 17– 19 follow the mesh into rendering and effects. Section 20–21 give measured costs and correctness invariants. Appendices contain the exact tables and storage layouts.

**The one-sentence summary**

The CPU decides *what the surface is*; the GPU decides *where its vertices are* and writes them once, directly into the buffers the renderer already owns.

## 2 Foundations and related work

### 2.1 Implicit surfaces and signed distance fields

A signed distance field assigns to every point $\mathbf{x} \in \mathbb{R}^3$ the signed Euclidean distance to the boundary $\partial\Omega$ of a solid $\Omega$:

$$f(\mathbf{x}) = \begin{cases} +\operatorname{dist}(\mathbf{x}, \partial\Omega), & \mathbf{x} \notin \Omega, \\ -\operatorname{dist}(\mathbf{x}, \partial\Omega), & \mathbf{x} \in \Omega. \end{cases} \tag{1}$$

The surface is the zero set $f^{-1}(0)$. The gradient of a true distance field has unit length and points away from the solid, so the unit normal is

$$\mathbf{n}(\mathbf{x}) = \frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}. \tag{2}$$

The engine stores only eight corner samples per octree cell and reconstructs $f$ inside the cell by trilinear interpolation. This is a deliberate approximation: it makes the field cheap to evaluate, cheap to cache, and identical between the CPU reference and the GPU shader.

### 2.2 CSG operations

A CSG operation combines two distance fields. The engine supports union, subtraction, intersection, and a generic user operation. For two fields $f_1, f_2$ the common rules are

$$\text{union}: \quad \min(f_1, f_2), \tag{3}$$

$$\text{intersection}: \quad \max(f_1, f_2), \tag{4}$$

$$\text{subtraction}: \quad \max(f_1, -f_2). \tag{5}$$

The exact expressions are not distance fields in the strict sense after subtraction (the result can be negative in the subtracted volume), but they preserve the zero set and the sign, which is all the mesher needs. The engine also supports smooth minima and custom effects; those change the field values but not the extraction algorithm.

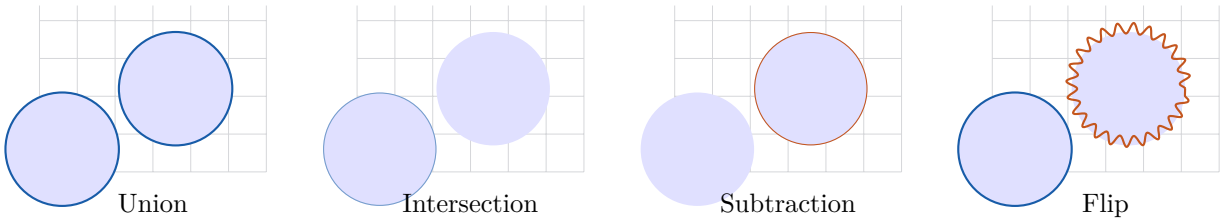



**Figure 2:** Two-dimensional cross-sections of the CSG rules. Blue is the solid; a subtraction removes the second field from the first; the rightmost panel illustrates a non-distance-preserving effect whose zero set and sign are still well defined.

### 2.3 Dual contouring and Surface Nets

Marching Cubes places one vertex on each sign-changing cube edge and connects them inside the cube. Dual methods instead place *one* vertex per surface cell and connect the vertices of cells that share an edge. The resulting mesh is the dual of the cell complex. Surface Nets use an average of edge crossings; Dual Contouring solves a quadratic error function (QEF) to place a vertex that fits the sampled Hermite data (position and normal on each crossing edge).

The engine uses a QEF solve (with a mass-point regularizer) to place one vertex per surface cell, then extracts triangles by scanning the twelve edges of each scan cell. The method is often called "dual contouring", but the extraction rule is the Surface Nets ring rule: for each active edge, collect the four cells around the edge, keep the surface cells, order them around the edge, and triangulate the resulting three- or four-gon. The QEF is what makes the vertex position smooth; the ring rule is what makes the connectivity exact and crack-free.

### 2.4 Adaptive octrees and LoD

A uniform dual grid at the finest resolution is wasteful: terrain near the camera needs centimetres, while a distant ridge needs tens of metres. An adaptive octree stores a cell only where the field crosses its cube and subdivides cells near the surface. Each cell has a "level"; a cell is allowed to be a leaf (no children) when its cube is small enough or when its eight children can be collapsed without exceeding an error threshold.

Adaptive dual meshing introduces two problems:

1. **The dual vertex of a cell can be far from its cube.** A QEF vertex is clamped inside its cell, but the four cells around an edge can have different sizes.
2. **Two neighbouring chunks can choose different levels.** A coarse cell on one side of a boundary and a fine cell on the other do not share the same edge vertices, so a naive fan leaves a crack.

The engine solves both with the same idea: for every active edge, sample the four quadrant cells at the edge midpoint and recursively collect *all* cell-boundary coordinates along the edge. The edge is split at those coordinates, and each sub-segment is emitted by its owner cell. A neighbouring chunk resolves the same edge with the same owner and the same sub-segment list, so the two sides agree bit-for-bit.

### 2.5 Why two processors?

The split used here is not a generic "CPU authoring, GPU rendering" division. It is driven by the representation:

- **The octree is a mutable, shared, pointer-rich structure.** It is edited by brushes, resampled

by shape operations, versioned, simplified, and guarded by a read/write lock. This is natural on the CPU and hostile to a GPU.
- **The triangle extraction is data-parallel and pure.** Given a cell's eight SDF samples and the four quadrant cells of an edge, the output is a deterministic function. This is ideal for a compute shader.
- **The renderer wants device-local buffers.** A CPU mesher produces a host array, which must be copied; a GPU mesher can write the renderer's buffers directly.

The cost is a flattening step. The host must traverse the octree neighbourhood and produce a table small enough to upload per chunk. The flattening is exact: it replays the same root-to-cell descent and the same child-index arithmetic the shader will use, so the GPU sees a *serialisable snapshot* of the tree, not a lossy approximation.

### 2.6 Position in the literature

The meshing rule descends from Gibson's Surface Nets and Ju et al.'s Dual Contouring. The adaptive stitching rule is related to the dual-cell "edge split" methods used in transvoxel-like terrain engines, but it is derived here directly from the CPU walker's break collection rather than from a fixed stitching table. The GPU port follows the same arithmetic order as the CPU reference (including the QEF's Cramer solve and the guarded trilinear lerp) so the two can be compared and the offline exporter can reproduce the runtime mesh.

## 3 The SDF model and its discretisation

### 3.1 Primitives and transformations

A scene is a list of operations applied to an initially empty field. Each operation carries a distance function, a transformation, a brush, and an optional painter. The distance functions are evaluated in the primitive's local space; the transformation maps the query point into that space and the result is scaled back when the field is not unit-speed. The engine's primitive catalogue includes spheres, boxes, capsules, cylinders, cones, tori, tapered primitives, height maps, road splines, swept polygons, and extruded SVG polygons. The exact primitive is irrelevant to the mesher: every primitive reduces to a function

$$d = f_{\text{primitive}}(\mathbf{x}_{\text{local}}), \tag{6}$$

and every CSG operation reduces to a scalar combination of two such values.

Brushes carry the material identity that eventually becomes the vertex's `brushIndex`; painters write the per-node HSV tint and the mapping mode/UV scale. The mesher never interprets either: it copies the node's brush and mapping state into the vertex payload in the geometry pass.

### 3.2 Corner sampling

The octree stores a field value only at the eight corners of each cube. For a cube with minimum $\mathbf{c}$ and edge $L$, corner $i$ is

$$C_i = \mathbf{c} + L\,\mathbf{s}_i, \qquad \mathbf{s}_i = \big((i \gg 2)\&1,\ (i \gg 1)\&1,\ i\&1\big), \tag{7}$$

with bit 2 selecting $x$, bit 1 selecting $y$, and bit 0 selecting $z$. The stored array is

$$\mathbf{d} = (d_0, \ldots, d_7), \qquad d_i = f(C_i). \tag{8}$$

A corner with no data is stored as $+\infty$. This sentinel is important: a value of $+\infty$ means "unknown", not "very far outside". The classifier skips it, and the QEF skips any edge with a non-finite endpoint. Treating it as a finite positive value would place vertices at the corner with a wild normal, creating the familiar "spike through the terrain" artefact.

### 3.3 Classification

The node type is computed from the signs of the finite corners:

$$\text{type}(\mathbf{d}) = \begin{cases} \text{Empty}, & \text{no finite negative corner}, \\ \text{Solid}, & \text{finite corners are all negative}, \\ \text{Surface}, & \text{both signs are present}. \end{cases} \tag{9}$$

The implementation is careful to skip $+\infty$ and to treat 0 as non-negative. A cell with a negative corner and an $+\infty$ corner is classified *Solid* by the stored type yet can still emit a crossing edge; therefore the GPU scan list is filtered by an explicit "straddles the surface" test, not by the stored type alone. This distinction is one of the subtle correctness points of the flattening.

### 3.4 Trilinear reconstruction and its gradient

Inside a cube, write the local coordinate

$$\mathbf{u} = \frac{\mathbf{x} - \mathbf{c}}{L} \in [0, 1]^3. \tag{10}$$

The trilinear interpolant is

$$\begin{aligned} f(\mathbf{u}) = {} & (1-u_x)(1-u_y)(1-u_z)d_0 + (1-u_x)(1-u_y)u_z d_1 \\ & + (1-u_x)u_y(1-u_z)d_2 + (1-u_x)u_y u_z d_3 \\ & + u_x(1-u_y)(1-u_z)d_4 + u_x(1-u_y)u_z d_5 \\ & + u_x u_y(1-u_z)d_6 + u_x u_y u_z d_7. \end{aligned} \tag{11}$$

The analytic gradient is used for normals:

$$\begin{aligned} \frac{\partial f}{\partial x} = {} & (1-u_y)(1-u_z)(d_4 - d_0) + (1-u_y)u_z(d_5 - d_1) \\ & + u_y(1-u_z)(d_6 - d_2) + u_y u_z(d_7 - d_3), \end{aligned} \tag{12}$$

$$\begin{aligned} \frac{\partial f}{\partial y} = {} & (1-u_x)(1-u_z)(d_2 - d_0) + u_x(1-u_z)(d_3 - d_1) \\ & + (1-u_x)u_z(d_6 - d_4) + u_x u_z(d_7 - d_5), \end{aligned} \tag{13}$$

$$\begin{aligned} \frac{\partial f}{\partial z} = {} & (1-u_x)(1-u_y)(d_1 - d_0) + (1-u_x)u_y(d_3 - d_2) \\ & + u_x(1-u_y)(d_5 - d_4) + u_x u_y(d_7 - d_6). \end{aligned} \tag{14}$$

The world-space gradient is $\nabla_{\mathbf{x}} f = (\partial_x f, \partial_y f, \partial_z f)/L$. The normal is the normalized gradient. If the gradient is degenerate, the shader falls back to $(0, 1, 0)$; this is preferable to a NaN vertex.

### 3.5 The trilinear lerp with sentinels

A naive `mix(a,b,t)` evaluates $a(1-t)+bt$. If one endpoint is $+\infty$ and the corresponding weight is exactly zero, the product is $0 \cdot \infty = \text{NaN}$. The engine uses a guarded lerp:

$$\text{lerp}(a,b,t) = \begin{cases} a, & t \le 0, \\ b, & t \ge 1, \\ +\infty, & a = +\infty \text{ or } b = +\infty, \\ a + (b-a)t, & \text{otherwise.} \end{cases} \tag{15}$$

This keeps the sentinel semantics through interpolation and is used by the CPU reference, the GPU shader, and the simplifier. It is also why the shader marks the sensitive expressions `precise`: reassociation can change the sign of an expression that is exactly zero at a corner.

### 3.6 The QEF vertex

For each finite sign-changing edge $e = (i, j)$, let

$$\mathbf{p}_e = (1-t_e)\mathbf{p}_i + t_e\mathbf{p}_j, \qquad t_e = \text{clamp}\left(\frac{d_i}{d_i - d_j}, 0, 1\right). \tag{16}$$

The edge normal is the interpolation of the two corner normals, normalized. The QEF is

$$E(\mathbf{x}) = \sum_e \left(\mathbf{n}_e \cdot (\mathbf{x} - \mathbf{p}_e)\right)^2. \tag{17}$$

Its normal equations are

$$\Big(\sum_e \mathbf{n}_e \mathbf{n}_e^T\Big)\mathbf{x} = \sum_e \mathbf{n}_e \left(\mathbf{n}_e \cdot \mathbf{p}_e\right), \tag{18}$$

solved by Cramer's rule. Two regularizers are essential:

1. the matrix is shifted by $\epsilon I$ with $\epsilon = 10^{-2}$;
2. the right-hand side is shifted by $\epsilon\,\bar{\mathbf{p}}$ where $\bar{\mathbf{p}}$ is the average edge crossing.

Without them, a cell with one crossing edge has a rank-one matrix and the solution can shoot arbitrarily far away. The result is clamped to the cell cube, so a dual vertex can never leave its cell.

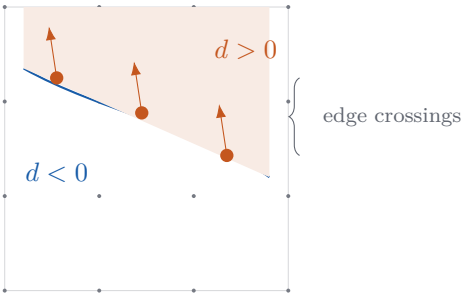



**Figure 3:** A surface cell in two dimensions. The shaded region is the inside ($d < 0$). Sign changes on cube edges give crossing points and normals; the QEF fits a dual vertex to those planes. In three dimensions there are twelve cube edges, and the vertex is clamped inside the cube.

### 3.7 Brushes, painters, and mapping state

The node stores the brush index, an HSV tint, the texture-mapping mode, and two generic data values. The brush is painted at the shape leaf; internal nodes inherit the dominant painted child. The GPU vertex payload packs:

| Word | Components | Source |
|---|---|---|
| 0 | position.xyz, color.x | QEF position; color is 1 |
| 1 | color.yz, texCoord.xy | texCoord = node data (UV scale) |
| 2 | normal.xyz, brushIndex bits | normalized SDF gradient; brush |
| 3 | textureType bits, hsv | mapping mode; HSV tint |

The renderer treats the 64-byte record as the canonical `Vertex`; the same record is used by the raster path, the ray-tracing BLAS, and the offline exporter. This is why the geometry pass does not need a conversion pass.

# 4 The CPU octree data structure

## 4.1 Node layout

An `OctreeNode` is a compact record:

| Field | Purpose |
|---|---|
| `float sdf[8]` | eight corner samples (may contain $+\infty$) |
| `int brushIndex` | material index, or the discard sentinel |
| `int textureType` | triplanar / projection / custom mapping |
| `glm::vec2 data` | mapping scale or authored UV |
| `glm::vec3 hsv` | tint carried to the vertex payload |
| `uint8_t lod` | stored ladder level, +1 shifted; 0 = unset |
| `uint8_t chunkLod` | chunk-rung level, +1 shifted; 0 = not a chunk |
| `uint8_t bits` | leaf flag, chunk flag, type |
| `uint blockId` | index of the child block in the allocator |
| `uint version` | bump on every published revision |

The +1 shift is a storage trick: a single byte can represent "unset" (0) and levels 0–254 without a separate sentinel. Every propagator uses the same convention:

$$lod_{\text{parent}} = \min_i lod_{\text{child},i} + 1, \qquad chunkLod_{\text{parent}} = \min_i chunkLod_{\text{child},i} + 1. \tag{19}$$

The GPU shader compares the stored level directly against the target level; no shift is applied in the shader.

Children are stored in a separate `ChildBlock`, an eight-entry array allocated from an `OctreeAllocator`. A leaf has no block; an internal node has a block with up to eight children. The allocator is a pool of blocks and nodes; freeing a subtree returns blocks to the pool and bumps the generation of the parent. Because node addresses are recycled, every publish/delete handler must be paired: a new node at a recycled address must not inherit the previous occupant's pending mesh. The engine's change handler explicitly reports the whole deleted subtree before `clear()` releases the children.

## 4.2 Cube corners and edges

The corner convention is bit-based:

| $i$ | $x$ | $y$ | $z$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 2 | 0 | 1 | 0 |
| 3 | 0 | 1 | 1 |
| 4 | 1 | 0 | 0 |
| 5 | 1 | 0 | 1 |
| 6 | 1 | 1 | 0 |
| 7 | 1 | 1 | 1 |

The twelve edges are

$$\begin{aligned}&(0,1),(1,3),(2,3),(0,2),\\&(4,5),(5,7),(6,7),(4,6),\\&(0,4),(1,5),(2,6),(3,7).\end{aligned} \tag{20}$$

This ordering is load-bearing: the QEF loop, the GPU scan loop, the Blender exporter, and the CPU reference all iterate the same twelve edges in the same order. Changing it changes the floating-point accumulation order and therefore the last bits of a vertex position.

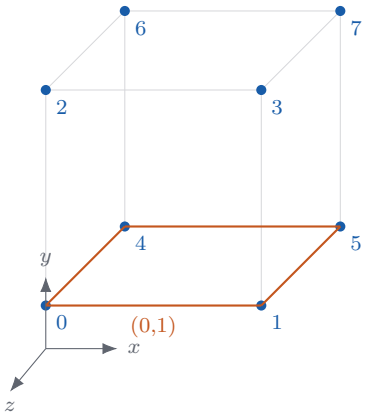



**Figure 4:** Corner numbering and the first four edges. The cube is drawn with $z$ receding; the corner bits are $(x, y, z)$ with bit $2 = x$, bit $1 = y$, bit $0 = z$. The GPU table stores corners in two `vec4`s and edges as twelve integer pairs.

### 4.3 Types and sentinels

Each node has a `SpaceType`: `Empty`, `Surface`, or `Solid`. The classification uses `SDF::eval`:

$$\text{eval}(\mathbf{d}) = \begin{cases}\text{Empty}, & \text{no finite negative corner and no finite positive corner},\\ \text{Solid}, & \text{all finite corners negative},\\ \text{Surface}, & \text{otherwise (both signs present)}.\end{cases} \tag{21}$$

An all-$+\infty$ cell is Empty. A cell with only positive finite corners is Empty. A cell with only negative finite corners is Solid. A cell with both is Surface. The edge scan is stricter: it tests the *sign pair* of each edge directly, so a surface cell whose only negative corner is diagonally opposite a positive one may have no active cube edge; the scan simply emits nothing.

### 4.4 Level of detail and chunk ladder

Two distinct levels live on a node:

**`lod`** the simplification ladder level. A leaf at the tessellation frontier has $lod = 1$; a parent whose eight children can be collapsed has $lod = \min child + 1$; the root has the largest value.

**`chunkLod`** the chunk rung. A node is a chunk root when its cube is between $\Delta/2$ and $\Delta$; its immediate descendants are marked $chunkLod = 1$, their children 2, and so on. The renderer's LoD gate consumes this value.

The chunk size $\Delta$ is fixed per scene (for example 512 world units). A chunk root at level 0 is the finest published rung; its ancestors form the coarser rungs. The renderer keeps exactly one rung per region visible, which is why the mesher publishes every rung rather than only the finest one.

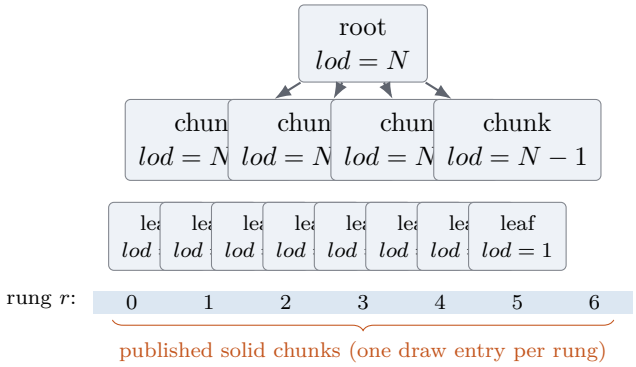



**Figure 5:** The chunk ladder. Every surface-bearing cell whose size lies in $(\Delta/2, \Delta]$ is a chunk root at some rung. Its children are the next finer rung, and so on. The GPU gate walks this dyadic lattice from the root to choose exactly one visible rung per region.

### 4.5 Memory model

The octree is a sparse pointer structure, not a dense grid. A node is small (the eight floats dominate) and a block is eight pointers. For a reference terrain with about $10^6$ surface cells the node storage is on the order of tens of megabytes, which is small compared with the GPU pools. The expensive part is not the octree itself but the per-chunk flattened upload: a chunk's neighbourhood can contain several thousand nodes, each uploaded as a 128-byte record (eight floats, cube, metadata, HSV/mapping, children). Section 20 measures this traffic.

### 4.6 Concurrency

`Octree::apply` takes a unique lock on `treeMutex`; every iteration (‘iterateMultiThreaded‘, the exporter walk, the debug cube walk) takes a shared lock. The shape recursion forks at a band of cube sizes and joins its children before returning, so a tree never exposes a partially written child block to a reader. The flattening runs under the shared lock: it reads the tree structure and copies the neighbourhood into a private `OctreeTopologyInput`. The GPU pass then works on the copy, so the tree can be edited again while the GPU is still meshing an older snapshot. The version recorded in the chunk manager (not the pointer) decides whether a rebuild is needed.

# 5 CPU shape construction: `Octree::apply`

## 5.1 Entry point and root expansion

An edit is expressed as

```
void Octree::apply(operation, function, model, painter, coordPainter,
                   minSize, simplifier, updateHandler, deleteHandler) {
    unique_lock treeLock(treeMutex);
    ShapeArgs args = ...;       // operation, function, model, painter, simplifier
    expand(args);               // grow the root if the new shape lies outside
    configureShapeCache(args);  // lattice origin/spacing for this apply
    OctreeNodeFrame frame(root, nullptr, *this, ...);
    NodeOperationResult r;
    shape(r, frame, args, &localContext, updateHandler, deleteHandler);
}
```

**Listing 1:** The public shape entry point (simplified).

`expand` grows the root by powers of two until the new primitive is fully contained. It computes the child index of the primitive's centre in the current root cube and places the old root as that child of a new, doubled root. This is the only operation that changes the world cube; the existing subtree is preserved bit-for-bit. The lattice origin and leaf length used by the SDF cache are computed *after* expansion.

## 5.2 The recursive `shape` call

For a cube of length $L$ and a target leaf size $m$, the recursion has three regimes:

1. **Leaf:** $L \leq m$ and the node is a leaf. Sample the eight corners of the shape field, combine them with the existing field, and classify the result.
2. **Interior with early out:** the node is not a leaf, but the shape field's centre value and bounding-volume test prove that the primitive cannot affect this cube. Copy the existing field and stop.
3. **Interior descend:** recurse into the eight children, join the results, classify the parent from the children, and optionally simplify.

## 5.3 Sampling and caching

The shape field is sampled at cube corners with `evaluateSDF`. A naive implementation calls the primitive's distance function once per query. The engine uses a per-apply cache keyed on the apply's leaf lattice:

$$\mathbf{x} = \mathbf{o} + \mathbf{n}\, h, \tag{22}$$

where $\mathbf{o}$ is the lattice origin and $h$ the half leaf length. An integer lattice key is accepted only when the reconstructed float is bit-identical to the query. Anything off-lattice (a stored tree from an earlier apply with a smaller leaf size) falls back to a float-keyed map. The cache is a pure optimisation: the returned value is the value the primitive would produce. The cache is per-thread, generation-tagged, and reset when the generation changes. This avoids both a global lock and stale values across applies.

## 5.4 The three early-out rules

The recursion has three important early outs. They are necessary for performance and for correctness at the field's infinity regions.

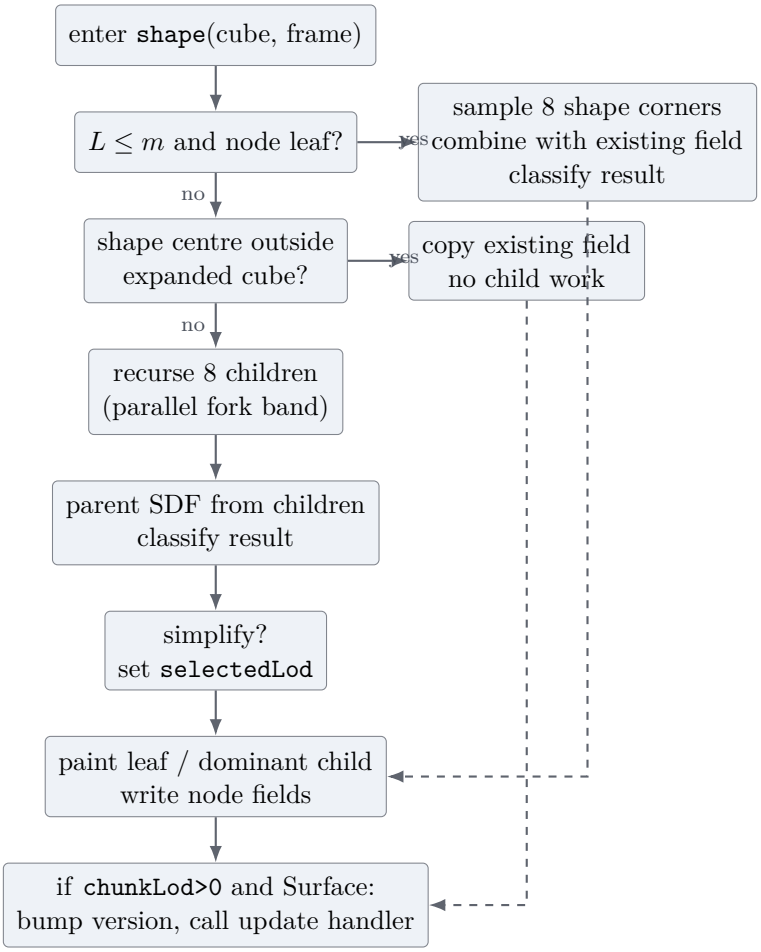



**Figure 6:** The shape recursion. A leaf samples the new field; an unaffected interior copies the old field; an affected interior recurses, classifies the parent from its children, simplifies, paints, and (for surface chunk roots) publishes a new version.

**All-infinity node.** If no corner of the existing field has data, the node is untouched. The operation may propagate from infinity (a union with a finite shape) or not (a subtraction whose negative operand is undefined). If the shape's centre is more than the cube's half-diagonal away from the solid, the whole cube is classified Solid or Empty without descending.

**Solid-preserving operation.** If the existing cube is Solid, all its corners finite, and the operation preserves solidity, the result is still Solid. The operation only needs to combine the existing and shape corner values; no child descent is needed. This is the common case for a union of overlapping solids.

**Solid cube and centre test.** If the existing cube is Solid and the combined centre is more than the cube's half-diagonal inside the surface, the whole cube is Solid. This avoids descending into regions the new shape cannot reach. The test uses the half diagonal $L\sqrt{3}/2$, not the half edge: a sphere can enter through a corner.

If none of the early outs applies, the function's `check(cube)` test is consulted. A `Disjoint` result skips the children; a `Contains` result can be handled by the same solid/empty rules. Finally, if the node is a new surface node with no existing children, the recursion is forced to descend even if the centre is outside, because the surface may still cross a corner.

### 5.5 Result composition

For a non-leaf node, each child returns eight result SDF values. The parent's result array is the concatenation of the children's corner arrays in child order. The parent's type is:

$$\text{parentType} = \begin{cases} \text{Empty}, & \text{all children Empty}, \\ \text{Solid}, & \text{all children Solid}, \\ \text{Surface}, & \text{otherwise}. \end{cases} \tag{23}$$

This is exact because the child corner arrays share the parent's corners on their common faces. The parent does not need to re-evaluate the field.

### 5.6 Painting

Painting happens at the shape leaf, where the node's material is decided. Brushes can be constant (a material index), procedural (terrain classification from height and slope), or position-dependent (the `LandBrush` classifier). The painter writes:

- `brushIndex` (material),
- `hsv` (tint),
- `textureType` (mapping technique),
- `data` (UV scale or authored UV).

Internal nodes inherit the dominant painted child's state. The exact dominance rule is intentionally simple: the last surface child with a valid brush that wins the per-child tally sets the parent. The tally is kept per child (not across all eight) so the historical result is preserved; changing it would change the mesh's material assignment and UV scale.

### 5.7 Node publishing and versions

A node that is both a surface and a chunk root (`chunkLod>0`) is a published chunk. At the end of its recursion, the engine:

1. sets its type, SDF, chunk flag, levels, brush, mapping state;
2. increments `version`;
3. calls the update handler with `(level, node, cube)`.

The handler queues a `PendingMeshData`. The renderer later flattens the neighbourhood, dispatches the GPU walk, and publishes the span. A version is the only identity the renderer trusts: the same node address may be reused by the allocator, so pointer identity alone is not a revision.

### 5.8 Threading

The recursion forks when the child cube length lies in

$$\left(\frac{m \cdot 16}{4},\ m \cdot 16\right], \tag{24}$$

where $m$ is the target leaf size. A forked node enqueues eight tasks into the octree's thread pool; each task owns a copy of its frame and the shape arguments and writes into a shared result block. The parent joins all eight futures before reading the block. The fork band is two levels deep: a $16m$ node forks eight $8m$ children and each of those forks eight $4m$ grandchildren; the $4m$ nodes do not fork again. This bounds the number of tasks per fork to 64 and avoids a thundering herd at the leaf level.

The join loop is careful about pool starvation. Instead of blocking on a future while the pool is idle, it calls `runOneTask()` to execute queued work on the waiting thread; only when the queue is empty does it block. This is what keeps a nested fork from deadlocking on a pool with fewer threads than the fan-out.

### 5.9 Why the CPU never solves a vertex

The current shape path deliberately avoids `SDF::getPosition` and `getNormalFromPosition`. The only host callers are the offline Blender exporter. This is not just a performance choice: it is a correctness boundary. If the runtime computed a vertex on the host and the GPU computed another, the two would differ in the last bits and the renderer's mesh would not match the exporter's. By making the GPU the sole Cartesian producer, the runtime has one answer and the exporter has a reference implementation that can be compared offline.

## 6 Simplification and the LoD ladder

### 6.1 What simplification means here

The octree's leaf lattice is the finest representation the shape pass ever produces. A coarser representation is obtained by removing all children of a node and treating its eight interpolated corners as the node's SDF. The node then becomes a leaf at a coarser level and its dual vertex is solved from the coarser eight samples. The mesh is still a valid dual mesh; it simply has fewer, larger cells.

Simplification is not a decimation of an existing triangle mesh. It is a decision made during the shape recursion, before any triangle exists. This is why the GPU can rebuild a chunk at any rung without a separate simplification pass: the tree already stores the chosen level.

### 6.2 The error criterion

For a candidate parent cube and its eight child results, the simplifier tests each surface child. For every corner of the child cube it computes the parent field's trilinear value and compares it with the child's stored corner value:

$$e = \max_{i \in \text{surface children}} \max_{j=0}^{7} \left| \text{interp}(\mathbf{d}_{\text{parent}}, C_{i,j}) - d_{i,j} \right| . \tag{25}$$

The parent may collapse only when $e \leq \tau L$, where $L$ is the parent edge and $\tau$ a user tolerance. A flat surface has small error and collapses; a cliff, a thin wall, or a sharp CSG seam has large error and stays subdivided.

```
SimplificationResult Simplifier::simplify(cube, sdf, children, chunkCube) {
    // 1. material consistency (texturing mode):
    //    all surface children must share one brush and be leaves
    // 2. chunk-border guard:
    //    never simplify a direct child of a chunk root
    // 3. re-interpolation error:
    //    for each surface child corner, |parent interp - child value|
    //    must be <= cube.getLengthX() * distance
    return result;
}
```

**Listing 2:** The simplifier's core checks (abridged).

### 6.3 The material and chain guards

Two non-geometric guards are essential:

1. **Material boundary.** If two surface children have different brushes, collapsing them would force one material onto the other. The simplifier refuses. This keeps a grass/rock boundary sharp even when the geometric error is small.
2. **Simplification chain.** A child may be collapsed only if it is itself a leaf. If a child still has children, collapsing the parent would skip a level of the ladder and break the hierarchy's monotonicity.

### 6.4 The chunk-border guard

A chunk root's direct children have length $\Delta/2$. They are the boundary cells between two chunks. If one chunk simplified such a cell and its neighbour did not, the two dual meshes would choose different edge vertices at the same boundary. The stitching algorithm can handle different cell sizes in the *interior* of the adaptive walk, but the chunk boundary is special: the two chunks are flattened independently and never share a node table. The guard therefore forbids simplifying the direct children of a chunk root. Coarser simplification is allowed only deeper in the subtree, where both sides of a boundary see the same neighbourhood and the break collector can stitch.

### 6.5 Level propagation

Each node's stored level is the minimum over its surface children plus one:

$$lod(v) = \begin{cases} 1, & v \text{ is a leaf at the frontier,} \\ 0, & v \text{ is not a surface node,} \\ \min_{c \in \text{children}(v)} lod(c) + 1, & \text{otherwise.} \end{cases} \tag{26}$$

The renderer uses the level as the rung index. The chunk's own level is propagated the same way but only along the chunk ladder:

$$chunkLod(v) = \begin{cases} 1, & v \text{ is a chunk root,} \\ \min_c chunkLod(c) + 1, & \text{otherwise.} \end{cases} \tag{27}$$

The published entry stores the 0-based `lod` (the shader/indirect gate subtracts one implicitly). A node with `chunkLod>0` and `resultType==Surface` gets a version bump and is published.

### 6.6 Why every rung is published

It is tempting to publish only the finest rung and let the renderer collapse it. That fails for two reasons:

- **The tree is sparse.** A coarse region may have no fine leaf at all; the only representation is the coarse node's eight samples.
- **The dual mesh is not a simple triangle subset.** A coarse cell produces a different dual vertex and a different ring than its eight fine children. Decimating the fine mesh is not equivalent to solving the coarse dual mesh.

Publishing every rung gives the GPU gate a complete ladder. The gate then selects exactly one rung per region, so the extra rungs cost memory but not draw calls. The memory is bounded by the number of chunk-bearing nodes (about one per $(\Delta/2, \Delta]$ cell) and is preallocated.

### 6.7 Chunk versions and rebuilds

Every published chunk carries a version. The scene layer records the version it has already emitted; a publish with the same version is `UpToDate` and is skipped. An edit bumps the version of the affected chunk roots and their ancestors. The renderer's pending queue then contains a new entry for each affected rung. The old span remains in the pool until the new span is published, so no frame ever sees a hole.

### 6.8 Cost model

Let $C$ be the number of surface cells at the finest lattice, $N$ the number of nodes in the tree, and $R$ the number of ladder rungs. The simplifier is linear in the number of surface children and corner tests:

$$T_{\text{simplify}} = \mathcal{O}(NR\,8 \cdot 8) \tag{28}$$

in the worst case, but in practice most nodes fail the first material or chain guard and do not reach the corner loop. The dominant cost of a shape apply is the field evaluation, not the simplifier. The cache makes repeated corner evaluations across neighbouring cubes cheap.

## 7 The CPU reference walker

The CPU walker is `OctreeTriangleTessellator::iterateTriangles`. It is no longer used at runtime, but it defines the semantics the GPU shader must reproduce and it is the only mesher used by the offline Blender exporter. Reading it is the best way to understand the GPU port because the shader is a line-by-line translation with an explicit stack instead of C++ recursion.

### 7.1 Frontier and scan cells

The walk starts at a chunk node and a target LoD $r$. It descends to the cells at that rung:

```
void walk(node, cube, targetLod) {
    if (node.lod == targetLod) {
        scanCell(node, cube);
    } else if (node.lod > targetLod) {
        for (child in node.children) walk(child, cube.child(i), targetLod);
    }
}
```

**Listing 3:** Frontier descent (conceptual).

A *scan cell* is a cube at the target rung. Only cells that straddle the surface need to be scanned. The GPU flattening compacts the list with the same "has negative and non-negative corner" test, because a cell with no sign change on any of its twelve edges cannot emit a triangle.

### 7.2 Twelve edges and edge spans

For a scan cell with cube minimum $\mathbf{c}$ and size $L$, each of the twelve edges is converted to an `EdgeSpan`:

- `axis`: the edge's varying axis (the one with the largest coordinate difference, which is exact for an axis-aligned cube);
- `fixedU`, `fixedV`: the two constant coordinates;
- `start`, `end`: the varying coordinate interval;
- `eps`: the geometric epsilon $\max(\text{worldSize}, L) \cdot 10^{-6}$;
- `sampleOffset`: $L \cdot 10^{-3}$, used to nudge boundary samples.

The span is active only when its two corner SDF values have opposite signs. The sign of the endpoint is also remembered; it is the authoritative winding oracle.

### 7.3 Why one edge can produce many segments

In a uniform grid, an active edge is shared by exactly four cells, all of the same size. The dual polygon is a quad (or a triangle when one of the four cells is not a surface cell). In an adaptive octree, the four quadrant cells around an edge can have different sizes and can be nested. The edge can cross several cell boundaries along its length. The correct mesh must split the edge at every such boundary; otherwise one side of the boundary would connect to a vertex that the other side does not know.

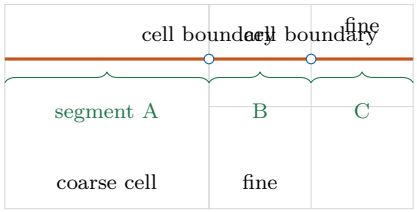



**Figure 7:** An active edge crossing a coarse/fine cell boundary. The edge is split at every cell boundary coordinate; each sub-segment is emitted by its owner cell. Both the coarse and the fine side collect the same set of boundaries, so the resulting rings agree along the shared edge.

### 7.4 Collecting breaks

The break collector is the heart of the adaptive stitching. Given an edge span and an interval $[a, b]$, it samples the four quadrant cells at the midpoint. Each valid cell contributes its cube's minimum and

maximum coordinate along the edge axis. If these coordinates lie strictly inside $[a, b]$, they become breakpoints. The interval is then recursively split at each breakpoint and the process repeats. The recursion stops when the interval is shorter than $2\epsilon$ or contains no new break.

The crucial detail is that the four quadrant cells are resolved by `findCellAt`, a root-to-cell descent that mirrors the octree's `getChildIndex` arithmetic. A sample is nudged by one ulp in the two fixed axes so that a point exactly on a cell face does not land in both cells; a small normal offset replaces the ulp when the value is zero (the next float is subnormal and may be flushed to zero). The four samples are:

$$(\text{fixedU} - \delta, \text{fixedV} - \delta), \tag{29}$$

$$(\text{fixedU} + \delta, \text{fixedV} - \delta), \tag{30}$$

$$(\text{fixedU} - \delta, \text{fixedV} + \delta), \tag{31}$$

$$(\text{fixedU} + \delta, \text{fixedV} + \delta). \tag{32}$$

The first sample is resolved with a full descent; the other three reuse it if they still lie inside its cube, which is exact because `findCellAt` is a pure function of position. This is the same optimization in the CPU walker and the shader.

### 7.5 The break list and the interval stack

The CPU collector uses recursive calls and a `std::vector<float>` break list. The GPU port replaces recursion with an explicit LIFO stack:

```
void collectBreaks(edge, start, end) {
    pushTask(start, end);
    while (top > 0) {
        pop(a, b);
        if (b - a <= eps*2) continue;
        mid = 0.5*(a+b);
        local = {a,b};
        cells = sampleQuadrants(edge, mid);
        for each valid cell:
            addBreakLocal(cell.min[axis], a, b);
            addBreakLocal(cell.min[axis]+cell.size, a, b);
        sortUniqueLocal();
        for interior local breaks: addBreakGlobal(...);
        push sub-intervals in reverse order;
    }
}
```

**Listing 4:** The GPU break collector (shape).

The CPU version appends all interior breaks before recursing into the sub-intervals. The GPU version reproduces that order by pushing the sub-intervals in reverse so the LIFO pop visits them left to right. The global break list is therefore identical up to floating-point ordering.

The fixed capacities are part of the contract:

| Buffer | Capacity | Meaning |
|---|---|---|
| `BREAK\_CAP` | 64 | final breaks per edge span |
| `LOCAL\_BREAK\_CAP` | 32 | breaks inside one interval |
| `TASK\_STACK` | 256 | pending intervals |
| `POS\_MEMO` | 8 | QEF position cache per invocation |

A cap overflow does not truncate silently. `noteOverflow()` sets a per-invocation flag and increments `overflowCount`; the host fails the chunk and retries it instead of drawing a hole.

### 7.6 Owner selection

For each sub-segment $[a, b]$, the owner is the surface quadrant with the smallest ordering key:

1. smaller cube size first;
2. then smaller $x$ minimum;
3. then smaller $y$ minimum;
4. then smaller $z$ minimum;
5. then smaller node index (a deterministic stand-in for pointer order).

The owner is computed from the four quadrants at the segment midpoint, not from the scan cell. This is what makes two neighbouring chunks agree: both see the same four quadrant cells and both choose the same owner.

### 7.7 Polygon reconstruction

The four quadrants are filtered to surface cells and deduplicated by node identity and by position (within the edge epsilon). A coarse ancestor can appear in two opposite quadrants of the ring; the old code only removed adjacent duplicates and then rejected the whole segment, which produced the missing surface along the world axes. The current code deduplicates against every already-collected quadrant. If the resulting list has three or four entries, it is rotated so the owner is first; then it is fanned into one or two triangles.

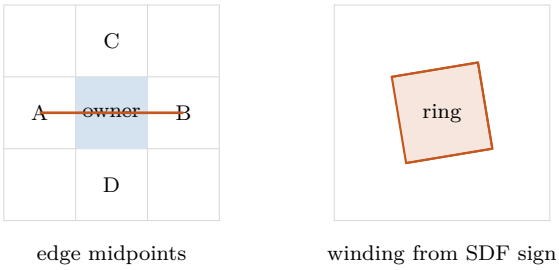



**Figure 8:** Left: the four quadrant cells around an edge. Right: the dual polygon is the ring of their vertices. The ring is rotated so the owner leads, then fanned. The winding is chosen from the SDF sign at the edge start, not from the normals.

### 7.8 Winding

The triangle order must be consistent with the SDF sign. The walker takes the interpolated SDF at the segment start using the owner's field. If the owner's interpolation is infinite or degenerates, it falls back to the producing cell's endpoint sign. A solid start means the ring is ordered so the outward normal points away from the solid; a positive start means the order is reversed. The sign is an *authoritative oracle*: it is independent of the QEF normal, which can degrade at coarse levels and on non-planar quads. This is why the surface does not flip at LoD transitions.

### 7.9 Triangle fan and duplicate rejection

A 3-ring produces one triangle; a 4-ring produces two, fanned around the owner. Before emission, the three dual positions are compared; if any two are within the edge epsilon, the triangle is dropped. A degenerate QEF vertex (or two cells that collapsed to the same position) must not create a zero-area triangle: it would break normals, shadows, and ray tracing.

### 7.10 The CPU triangle handler

The CPU walker passes each triangle to `Tesselator::handle`, which:

- resolves the three cell vertices to `Vertex` records;
- copies brush, HSV, texture type, and mapping data;
- computes the triplanar UV plane from the face normal;
- appends the triangle to the chunk's `Geometry`.

The GPU port does not have a `Tesselator` equivalent. The geometry pass writes the vertex payload directly from the emitting node and appends the index stream directly into the chunk's span. This is possible because the CPU handler's output is a pure function of the node state.

### 7.11 Why keep the CPU walker?

The CPU walker is retained for three reasons:

1. **Offline export.** The Blender exporter has no GPU and must produce the same mesh.
2. **Reference.** The GPU shader's arithmetic can be compared chunk-by-chunk with the CPU walker under a self-check flag.
3. **Debugging.** When a GPU mesh is wrong, the CPU walker can generate the expected triangles for a small chunk and the two sets can be diffed.

It is not a fallback: the runtime never invokes it. A chunk that cannot be flattened is skipped, not CPU-meshed.

## 8 GPU-resident meshing: design constraints

### 8.1 The contract

The GPU mesher must satisfy a strict contract:

**G1. No host vertex stream.** The host may upload node tables, scan-cell indices, and small counters. It must not upload or download a Cartesian vertex.

**G2. Renderer layout.** The output vertex must be the renderer's 64-byte `Vertex`; the output index must be a chunk-local slot index that the renderer's draw command can rebase with `vertexOffset`.

**G3. Bit-exact parity with the CPU reference.** The same chunk must produce the same triangle set and the same QEF positions as the CPU walker, up to the documented floating-point ordering.

**G4. Deterministic slots.** The host must assign vertex slots in a stable order so a rebuild produces the same pool layout and the ray-tracing BLAS does not see gratuitous changes.

**G5. Bounded work and memory.** A pathological chunk must fail explicitly (overflow/drop counter) rather than corrupt the pool or draw a partial surface.

### 8.2 Why one geometry pass?

The walk output is not known before the scan: any active edge can emit any number of sub-segments, and each sub-segment expands to one or two triangles. A two-stage design solved this by writing one raw node key per triangle corner, reading the count back, reserving the exact index span, and remapping the keys in a second dispatch. That intermediate stream is unnecessary once the host can bound the output from the topology it already has.

While the scan-cell list is built, the host counts the chunk's *cross edges*: the `(cell, cube-edge)` pairs whose two stored corner samples have opposite signs. The shader scans exactly those edges, so the count matches its work list. The index capacity is then a topology bound, not a calibrated multiplier:

$$I_{\text{cap}} = \sum_{\text{scan cell } c} 6 \cdot \text{segments}(c) \cdot \text{crossEdges}(c), \tag{33}$$

where a segment expands to at most two triangles of three indices and

$$\text{segments}(c) = \text{clamp}\left(\frac{\text{cellSize}(c)}{\text{smallestFlattenedCellSize}}, 1, 64\right). \tag{34}$$

Every break coordinate the collector can insert is a boundary of a cell returned by `findCellAt`; all such cubes are aligned to the smallest cube in the flattened table, so one edge of size $S$ can contain at most $S/m$ dyadic intervals. The clamp is the shader's own break-list capacity.

The vertex capacity is exactly the dense surface-node slot address space `activeVertexCount`. The host assigns one slot per surface node in table order, and every index is one of those slots. The shader writes only the slots a triangle references, so the published extent can be smaller than the reserved capacity, but the address space itself is an exact upper bound.

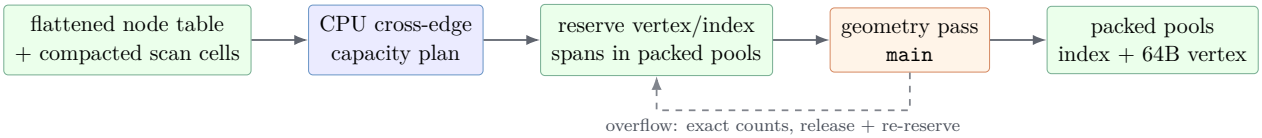



**Figure 9:** The single geometry dispatch. The CPU sizes both spans from the cross-edge topology before submission; the shader appends indices directly and writes each referenced vertex payload once. A defensive overflow is reported through counters and resolved by growing only this chunk's span.

### 8.3 Overflow without out-of-bounds writes

The shader still reserves an index base with `atomicAdd` and writes the triangle only when

$$\text{base} + 3 \le I_{\text{cap}} \quad \wedge \quad \text{extent} \le V_{\text{cap}}. \tag{35}$$

On failure it increments an overflow counter and drops the triangle, but it has already advanced the index counter by three and updated the maximum referenced slot. The host therefore reads back the exact index requirement and vertex extent without any write outside the reserved span. It releases that chunk's span and asks the provider for the grown capacities; the retry is limited to this chunk and normally the first grown attempt succeeds. A fixed-array overflow or a corner without a host-assigned slot is not retryable: that indicates a flattening disagreement, so the chunk is failed.

### 8.4 Determinism

The walker uses atomics for the index append, the overflow counters, and the per-node vertex election. Triangle order at the end of the index stream is therefore non-deterministic when two scan cells emit concurrently. Does that matter? The rendered triangle *set* is deterministic, and each corner carries its host-assigned slot. The renderer does not care about triangle order. The vertex slots are assigned by the host in node-table order, not by emission order, so a rebuild produces the same vertex buffer for the same node table. Ray tracing and shadow maps are order-independent. The only observable order is the index stream, which is internal to the chunk.

### 8.5 Precision strategy

The shader is a direct translation of the CPU arithmetic:

- corner order, edge order, and QEF accumulation order are identical;
- the trilinear lerp uses the same guarded branch;
- the QEF uses the same Cramer solve and the same $10^{-2}$ regularizer;
- the gradient and QEF use `precise` to prevent FMA contraction;
- the sample nudge uses `nextafter` semantics, including the subnormal guard at zero.

The goal is not merely a visually similar mesh; it is a mesh that the offline exporter can compare triangle-by-triangle. Where the GPU cannot guarantee the same order (atomic append), the triangle set is still identical because the walker's per-cell output is independent.

## 9 Flattening the octree neighbourhood

### 9.1 What the shader needs

A chunk's scan cell can query a neighbouring cell at a coarser or finer LoD, possibly in a neighbouring chunk. The shader cannot walk pointers, so the host must provide a node table that contains every cell the walk can reach. The table must include:

1. the world root at index 0 (the shader's descent anchor);
2. the root-to-chunk ancestor path, with root-consistent cubes;
3. all descendants of the chunk down to the target LoD;
4. all coarser nodes in the one-cell margin around the chunk, plus their children, because an edge sample can cross the boundary and land in a coarser neighbour;
5. a child index per node (or $-1$ when absent);
6. one dense vertex slot per surface node in the table.

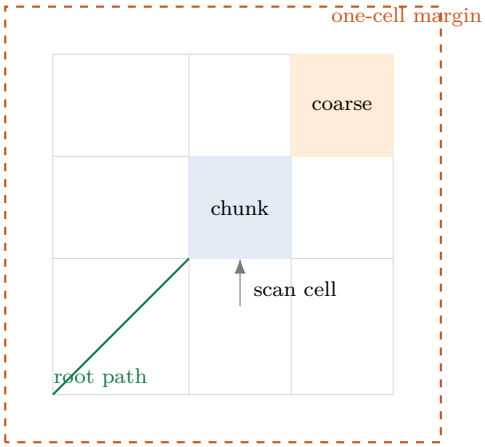



**Figure 10:** The flattened neighbourhood. The chunk subtree is walked to the target rung; the one-cell margin is included so edge samples that cross the boundary can resolve coarser neighbours. Index 0 is always the world root, because the shader's descent starts there for samples outside the chunk anchor cube.

### 9.2 Root-consistent ancestor chain

The host replays the child index from the world root to the chunk:

```
cube = rootCube; node = root;
chain.push({node, cube});
```

```
while (node != chunk) {
    i = cube.getChildIndex(chunkCenter);
    node = children[i];
    cube = cube.getChild(i);
    chain.push({node, cube});
}
```

**Listing 5:** Ancestor chain replay.

The replay is necessary because a cached chunk cube may have accumulated float drift. The shader will compute its own child indices from the world root, so the host must use exactly the same arithmetic. If the chain cannot be replayed (stale node, missing child), the chunk is marked `Unavailable`; there is no CPU fallback.

### 9.3 Descending the table

The table is built from the chain root. For each node, the host writes:

| Buffer | Contents |
|---|---|
| `sdf0, sdf1` | two `vec4`s of corner samples |
| `cubeMinSize` | `vec4(cubeMin, cubeSize)` |
| `meta` | `ivec4(lod, type, brush, flags)` |
| `hsv` | `vec4(hsv, textureType)`, `vec4(data,0,0)` |
| `child03, child47` | two `ivec4`s of child indices |
| `cells` | compacted scan-cell node ids |

The layout is 128 bytes per node: $4 \times 4$ floats for SDFs, 4 floats for the cube, 4 ints for metadata, 8 floats for HSV/mapping, and 8 ints for children. It is uploaded into host-visible coherent buffers; the shader reads it through storage-buffer descriptors.

### 9.4 The scan-cell list

The scan list contains cells exactly at the target rung that have at least one crossing edge. The host counts the crossing edges per cell while it builds the list:

$$\text{crossEdges}(c) = \#\{(i,j) \in \text{SDF_EDGES} : \text{sign}(d_i) \neq \text{sign}(d_j)\}, \tag{36}$$

with $+\infty$ counting as non-negative. This is not the same as `type == Surface`: a negative-plus-infinity cell is typed Solid yet can emit an edge. Filtering by type would silently drop those edges.

The same pass produces the index-capacity hint used by the geometry dispatch:

$$I_{\text{hint}} = 6 \sum_{c \in \text{cells}} \text{clamp}\left(\frac{\text{cellSize}(c)}{m}, 1, 64\right) \cdot \text{crossEdges}(c), \tag{37}$$

where $m$ is the smallest cube in the flattened table. A segment expands to at most two triangles of three indices; the clamp is the shader's break list capacity. This is a topology bound, not a statistical estimate. If it is nevertheless exceeded, the shader reports the exact requirement and the host grows only that chunk's span.

### 9.5 The descent anchor

The shader's `findCellAt` descends from the root. A per-sample root descent would be expensive, so the host supplies `fromId`: the first node on the chunk path whose stored LoD is at most the target LoD. For any sample inside that node's cube, starting the descent there is bit-identical to starting at the root, because no ancestor of the anchor is a stop node. Samples in the margin outside the anchor fall back to the root. This is a pure optimization; the result is the same.

### 9.6 Why the margin is one cell

An edge sample point is a corner of a scan cell, nudged outward by one ulp in two axes. The neighbouring quadrant cell can be one cell larger than the scan cell (a coarse ancestor). The flattening domain is therefore

$$[\mathbf{c}_{\min} - \ell,\ \mathbf{c}_{\max} + \ell], \tag{38}$$

where $\ell$ is the largest target-rung cell size in the subtree, *including non-surface cells that were compacted out of the scan list*. Deriving the margin from the compacted list would under-estimate the largest neighbour and could omit a cell the walk samples.

### 9.7 Vertex slots

Every node whose `type` is `Surface` receives a dense slot in table order:

$$\text{flags} \mid= (s + 1) \ll 1. \tag{39}$$

Bit 0 remains the leaf flag. The shader decodes

$$s = (\text{flags} \gg 1) - 1. \tag{40}$$

The count is the dense slot address space, an exact upper bound on every index the shader can write. It is not the number of vertices actually produced: the geometry pass writes the payload only for slots referenced by a triangle, and the published span is trimmed to the highest referenced slot plus one. Unreferenced slots between zero and that extent remain holes in the reserved span, so the per-chunk profile reports both the written vertex count and the published extent.

### 9.8 Table order and determinism

The table order is fixed by the traversal: root, chunk subtree in child-index order, then the margin descent. The slot assignment follows that order. A rebuild with the same tree produces the same slots. The renderer relies on this for the packed-pool span and for the ray-tracing BLAS, which references the same base vertex and index ranges.

### 9.9 Failure modes

Flattening can fail for only a few reasons:

- the chunk node is not reachable from the root (stale parameters);
- the chain is deeper than 200 (a corrupt tree);
- the chunk has no crossing scan cell, or no surface node in the flattened table owns a slot (a valid empty chunk, not a failure);
- the table is missing a child block the descent needs.

The first, second, and fourth mark the chunk `Unavailable`; the caller skips it and logs. The third returns a valid empty input, the geometry pass is not dispatched, and the publisher settles the empty revision.

# 10 The GPU geometry pass

## 10.1 Invocation model

The geometry dispatch has one invocation per compacted scan cell:

$$\text{groups} = \left\lceil \frac{|\text{cells}|}{64} \right\rceil, \tag{41}$$

with a local size of 64. The shader starts from `gl_GlobalInvocationID.x`; if it is beyond the cell count, it returns. All per-invocation state is private:

- break list (64 floats),
- local break list (32 floats),
- task stack (256 intervals),
- QEF position memo (8 entries),
- overflow flag and diagnostic counters.

No workgroup shared memory is needed. This makes the pass portable across devices with small shared-memory budgets and avoids workgroup synchronization.

## 10.2 Node accessors

The shader wraps the eight storage buffers in typed accessors:

```
float nodeSdf(uint id, int i) { return (i < 4) ? sdf0[id][i] : sdf1[id][i-4]; }
int nodeType(uint id)   { return meta[id].y; }
int nodeBrush(uint id)  { return meta[id].z; }
int nodeSlot(uint id)   { return int(meta[id].w >> 1) - 1; }
vec3 nodeMin(uint id)   { return cubeMinSize[id].xyz; }
float nodeSize(uint id) { return cubeMinSize[id].w; }
int nodeChild(uint id, int i) { ... }
```

**Listing 6:** Node accessors (abridged).

The table is a flat array; there are no pointers. The accessors are inlined by the compiler.

## 10.3 `findCellAt`

The descent mirrors the CPU walker:

```
Cell findCellAt(vec3 pos) {
    if (!worldContains(pos)) return INVALID;
    uint idx = pc.fromId >= 0 ? pc.fromId : 0;
    if (pos outside anchor cube) idx = 0;
    while (!nodeIsLeaf(idx) && nodeLod(idx) > pc.targetLod) {
        ivec3 half = nodeMin(idx) + 0.5*nodeSize(idx);
        int child = (pos.x >= half.x ? 4 : 0)
                  + (pos.y >= half.y ? 2 : 0)
                  + (pos.z >= half.z ? 1 : 0);
        int c = nodeChild(idx, child);
        if (c < 0) return INVALID;
        idx = c;
    }
    return Cell(idx, nodeMin(idx), nodeSize(idx));
}
```

**Listing 7:** The root-to-cell descent.

The stop condition is "leaf or stored LoD at most target". The stored LoD is +1 shifted, exactly as on the CPU. Starting at `fromId` is valid only for positions strictly inside the anchor cube; a point on the upper face must follow the sibling branch, so the test uses a strict upper bound.

### 10.4 Edge span construction

For each of the twelve cube edges, the shader loads the two corner signs. If they are equal, the edge is inactive. Otherwise it builds an `EdgeSpan`. The varying axis is the component with the largest absolute corner difference (exact for an axis-aligned cube). The two other axes are the fixed $u, v$ axes. The span's epsilon is

$$\epsilon = \max(\text{worldSize}, L) \cdot 10^{-6}, \tag{42}$$

and the boundary sample offset is

$$\delta = \max(L \cdot 10^{-3}, 10^{-6}). \tag{43}$$

The epsilon is used for duplicate rejection and sign-change tests; the offset is used when a sample sits exactly on a cell face.

### 10.5 The ulp nudge and the zero guard

Sampling a point exactly on a cell face is ambiguous: the two cells both claim it. The shader nudges the fixed coordinates by one float in the direction of the quadrant. At zero, `nextafter(0,+inf)` is a subnormal; FTZ GPUs flush it to zero, and the negative quadrant then resolves into the positive cell. The shader detects a subnormal nudge and uses a small normal offset instead. This is a small detail with a large visual consequence: without it, the surface disappears along the world $x = 0$, $y = 0$, and $z = 0$ planes.

### 10.6 The scan loop

For each active edge the walk:

1. initializes the global break list with the span endpoints;
2. calls `collectBreaks`;
3. sorts and uniques the global list;
4. iterates consecutive breaks and calls `emitSegment` for each interval wider than $2\epsilon$.

This is exactly the CPU loop. The only difference is that the CPU stores the breaks in a `std::vector` and the GPU uses fixed arrays.

### 10.7 Diagnostics

Two counters are accumulated only when `VULKAN_SETUP_DIAG` is set: `findCellCalls` and `reuseHits`. They are summed locally and published with one atomic pair per scan cell, so the diagnostic path does not add a per-call atomic. The normal path has a predictable branch and no counter traffic.

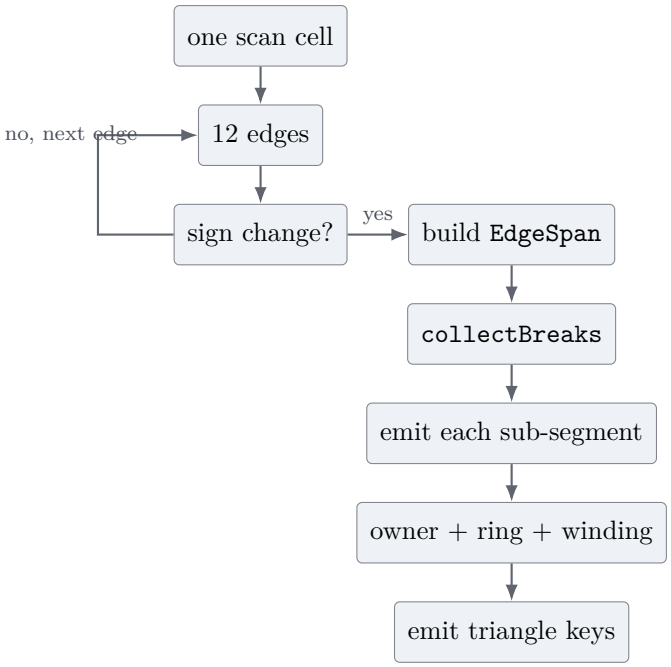



**Figure 11:** One scan-cell invocation. Inactive edges are skipped; active edges are stitched and emitted. The entire control flow is private to the invocation.

# 11 Step-by-step adaptive stitching

## 11.1 The exact problem

Consider an active edge shared by four cells. In a non-adaptive mesh, the four cells all have the same size and the edge contributes one segment. In an adaptive mesh, the four cells can be a mix of coarse and fine cells. The edge can cross a boundary between two cells at any coordinate. The correct mesh splits the edge at every such boundary and emits one segment per sub-interval. The two chunks on either side of a chunk boundary must produce the same split list; otherwise their rings disagree and a crack opens.

## 11.2 Why a fixed stitching table is not used

A transvoxel-style table enumerates the finite set of cell configurations and gives a triangulation for each. That works when the LoD difference across a boundary is at most one level and the cell sizes are known. Here the difference can be arbitrary (a coarse root cell next to a fine leaf), the tree is sparse, and the cell sizes are continuous. A table would have to enumerate a combinatorial set of nested cells. The break collector instead computes the split from the actual cell boundaries, which is configuration-independent.

## 11.3 The recursion, formally

Let $E$ be an edge span and $[a, b] \subseteq E$ an interval. Define

$$B(a,b) = \{\, c_x \mid \text{quadrant cell } c \text{ at } (a+b)/2 \text{ contributes } c_{\min,x} \text{ or } c_{\max,x} \,\} \tag{44}$$

restricted to values strictly inside $(a, b)$ and not within $\epsilon$ of an existing break. Then:

1. append every interior value of $B(a, b)$ to the global break list;
2. for each consecutive pair $a = t_0 < t_1 < \cdots < t_k = b$, recurse on $[t_{i-1}, t_i]$ if it is longer than $2\epsilon$.

The recursion terminates because each level inserts at least one break, or the interval is shorter than $2\epsilon$. In practice it terminates after one or two levels: the cell boundaries are powers of two and the interval is the current edge segment.

### 11.4 The GPU stack

The CPU implementation uses recursion. The shader replaces it with a LIFO stack of intervals. The CPU's exact order is:

- append all interior breaks first;
- then recurse into the sub-intervals from left to right.

The shader reproduces this by pushing sub-intervals in reverse order:

```
for (int i = localCount - 1; i >= 1; --i)
    pushTask(local[i-1], local[i]);
```

**Listing 8:** Reverse push preserves the CPU order.

Because the stack pops the last pushed interval first, it visits $[t_0, t_1]$ before $[t_1, t_2]$, and fully expands each before the next. The global break list is identical to the CPU's.

### 11.5 The sample offset

The quadrant samples are nudged by $\delta$ in the two fixed axes. The nudge is not a random jitter: it is a deterministic one-ulp move toward the quadrant. The two axes are chosen by `edgeAxes`: for an $x$-edge the fixed axes are $y, z$; for a $y$-edge they are $x, z$; for a $z$-edge they are $x, y$. The four quadrant signs are fixed by the `quadrantSigns` table:

| axis | $q_0$ | $q_1$ | $q_2$ | $q_3$ |
|---|---|---|---|---|
| $x$ | $(-1,-1)$ | $(-1,+1)$ | $(+1,+1)$ | $(+1,-1)$ |
| $y$ | $(-1,-1)$ | $(+1,-1)$ | $(+1,+1)$ | $(-1,+1)$ |
| $z$ | $(-1,-1)$ | $(-1,+1)$ | $(+1,+1)$ | $(+1,-1)$ |

The actual table is flattened as `QS[axis*8 + quadrant*2 + {0,1}]`; its values are shown above.

### 11.6 Break capacities and overflow

The fixed capacities are a hard contract. A scan cell with more than 64 breakpoints, or an interval with more than 32 local breaks, or a recursion stack deeper than 256 intervals is not expected for the engine's cell sizes, but it is still possible for a pathological SDF. The shader does not truncate:

- `noteOverflow()` sets `gOverflow`;
- the cell stops emitting;
- `overflowCount` is incremented;
- the host sees the counter and fails the chunk.

The host retries the chunk after the tree edit or reports it as skipped. It never publishes a partial index stream. This is an important difference from a "best effort" GPU mesher: a silent truncation would be a hole in the terrain that is hard to diagnose.

### 11.7 The global break list

The global list is sorted and deduplicated by insertion sort and a linear unique pass. The lists are small (at most 64 entries), so an $\mathcal{O}(n^2)$ sort is faster than a library sort and, more importantly, has

a deterministic order. Equality uses the edge epsilon:

$$|b_i - b_j| \leq \epsilon \quad \Rightarrow \quad \text{duplicate.} \tag{45}$$

The sort is stable for equal keys because the insertion sort only moves strictly greater values. This matches the CPU's `std::sort` behaviour for these list sizes and keeps the emitted triangle order deterministic.

### 11.8 Owner and quadrant sampling

For each sub-segment the shader samples the four quadrants at its midpoint. It does not reuse the quadrant cells from the break collection: the midpoint of a sub-segment is a different position and may fall in a different cell. The owner is selected by the ordering rule of Section 7. The owner's interpolated field at the segment endpoints decides the winding. If the owner's interpolation degenerates (both endpoints get the same sign because a zero lies exactly on a corner), the producing cell's endpoint signs are used instead. Those signs are authoritative because the producing cell only scans edges with a sign change.

### 11.9 Consequences

The stitching step is what makes it legal to publish every LoD rung and let the renderer choose one per region. Without it, the mesh would be crack-free only within a single rung; at rung transitions the surface would have T-junctions and visible seams. With it, the coarse and fine sides of a boundary agree on the edge split and on the owner, so their duplicated boundary quads have identical vertex order and identical triplanar plane. This is also why the chunk-border guard in the simplifier is necessary: the stitching can resolve interior LoD changes, but it cannot resolve a difference in the chunk's own boundary cells because the two chunks never share a node table.

## 12 Polygon reconstruction, winding, and triangle emission

### 12.1 From four quadrants to a ring

For a sub-segment the shader resolves four quadrant cells. Not all of them are surface cells: a coarse cell can have no zero crossing at the sample. The algorithm:

1. iterate quadrants in the fixed order $q_0, \ldots, q_3$;
2. skip non-surface cells;
3. compute the cell's dual vertex position with the memoized QEF;
4. reject a cell if its node id or position duplicates an already collected cell (within the edge epsilon);
5. stop at four collected cells.

The deduplication is not merely an optimization. A coarse ancestor can appear in two opposite quadrants of the ring, especially on world-axis cell boundaries where the sample nudge stays inside the same large cell. The old code removed only adjacent duplicates and then aborted the whole segment if any duplicate remained; that is exactly why triangles were missing along the $x = 0$, $y = 0$, and $z = 0$ planes. Deduplicating against all collected cells keeps the remaining 3 or 4 corners and preserves their order.

### 12.2 Rotating the owner to the front

The ring is rotated so the owner cell is first:

```
find k with ring[k].id == owner.id or ring[k].pos == owner.pos;
if (k > 0) rotate ring left by k;
```

**Listing 9:** Owner rotation.

Both chunks on either side of a boundary resolve the same owner (the ordering key is global), so their duplicated boundary quads get the same rotation. Without this, the same quad would be triangulated from different anchors on the two sides, producing different triplanar planes and z-fighting along the seam.

### 12.3 Three-ring and four-ring fans

A three-ring emits one triangle:

$$(r_0, r_1, r_2). \tag{46}$$

A four-ring emits two:

$$(r_0, r_1, r_2), \quad (r_0, r_2, r_3). \tag{47}$$

If the owner's start sign is positive (outside), the order is reversed:

$$(r_0, r_2, r_1), \quad (r_0, r_3, r_2). \tag{48}$$

This is a fan around the owner. A ring with fewer than three cells emits nothing; it is a boundary case where the surface only touches one or two quadrants.

### 12.4 The winding oracle

The winding is not derived from the triangle normal. It is derived from the SDF sign at the segment start, evaluated with the owner's trilinear field:

$$\text{solidAtStart} = \big(d_{\text{owner}}(a) < 0\big). \tag{49}$$

If the owner's interpolation returns $+\infty$, the producing cell's endpoint sign is used. The producing cell is the cell that scanned the edge, and it only scans edges whose own corner signs differ; those signs are therefore authoritative for existence.

Why not use the QEF normal? At coarse LoD a cell can have only one or two crossing edges, and the QEF normal is poorly constrained. A triangle wound from a noisy normal can flip at a LoD boundary, making the surface black or invisible from one side. The sign oracle is independent of the normal and robust across cell sizes.

### 12.5 Degenerate triangles

Before appending, the shader compares the three dual positions:

$$\|p_i - p_j\| \le \epsilon \quad \Rightarrow \quad \text{drop}. \tag{50}$$

This catches two cases:

- two cells collapsed to the same QEF position (a thin sliver);
- a cell with an exact-zero corner snapped to the corner, coinciding with a neighbouring cell's vertex.

A zero-area triangle would break normals, shadow-map rasterization, and ray-triangle intersections. Dropping it is safer than emitting it.

### 12.6 Direct index append and lazy vertices

The triangle is appended straight into the chunk's index span:

```
uint base = atomicAdd(indexCount, 3u);
int s0 = nodeSlot(c0.id), s1 = nodeSlot(c1.id), s2 = nodeSlot(c2.id);
if (s0 < 0 || s1 < 0 || s2 < 0) { atomicAdd(invalidSlotCount, 1u); return; }
uint extent = max(max(s0, s1), s2) + 1u;
atomicMax(vertexExtent, extent);
if (base + 3u > pc.indexCapacity || extent > pc.vertexCapacity) {
    atomicAdd(indexOverflow, (base + 3u > pc.indexCapacity) ? 1u : 0u);
    atomicAdd(vertexOverflow, (extent > pc.vertexCapacity) ? 1u : 0u);
    atomicAdd(droppedCount, 1u);
    return;
}
poolIdx[pc.indexBase + base + 0u] = uint(s0);
poolIdx[pc.indexBase + base + 1u] = uint(s1);
poolIdx[pc.indexBase + base + 2u] = uint(s2);
writeVertexOnce(c0.id, s0);
writeVertexOnce(c1.id, s1);
writeVertexOnce(c2.id, s2);
```

**Listing 10:** Index append.

The atomic only reserves the base; the three index writes are private. The counter advances by three even when the triangle does not fit, so the host reads the exact index requirement for the overflow-growth retry. No write below the capacity check is ever outside the reserved span, so an overflow cannot corrupt another chunk.

The vertex payload is a pure function of its node. Many corners reference the same surface node, so the shader elects one writer per node with an `atomicExchange` on the device-local `nodeWritten` buffer, which is cleared before every dispatch. The elected invocation writes the 64-byte payload and increments `vertexWriteCount`; the others skip it. By the time the dispatch completes, every referenced payload is therefore written exactly once.

### 12.7 Triangle order and visual invariance

The order of triangles in the index stream is the order in which scan cells happen to append. Different GPUs, workgroup schedules, or occupancy can produce a different order. This is acceptable because:

- the triangle set is identical (each cell's output is independent);
- the renderer draws the whole index range, not a prefix;
- the shadow map, depth prepass, and ray tracing are order-independent;
- the vegetation and fire anchors are keyed by chunk/node, not by triangle index.

The only place order matters is debugging: a diff of two index streams must be order-insensitive. The engine's self-check compares triangle sets, not raw arrays.

### 12.8 The boundary quad

A duplicated boundary quad is emitted by both neighbouring chunks. Both copies must be identical in position, order, and material. The engine guarantees this by:

1. using the same owner ordering key (size, min, node index);
2. sampling the same four quadrants (the neighbour's node table contains the same cells);
3. using the same QEF arithmetic and the same edge epsilon;
4. rotating the owner to the front.

The duplicate triangles are not removed at the chunk level; the renderer draws both. They coincide exactly, so the depth test resolves the z-fight. Removing them would require a cross-chunk dedup

pass and would reintroduce a seam if the dedup key differed.

# 13 Step-by-step QEF vertex solve and payload

## 13.1 Why a QEF instead of an average

A Surface Nets vertex is often placed at the average of the edge crossings. That is cheap but smooths sharp features: a box corner becomes a rounded blob. A QEF fits a point to the tangent planes of the crossings, which preserves sharp edges better while still producing one vertex per cell. The engine regularizes the QEF toward the mass point so an underconstrained cell does not explode.

## 13.2 The twelve crossings

For each edge $(i, j)$ with finite endpoints and opposite signs:

$$t = \text{clamp}\left(\frac{d_i}{d_i - d_j}, 0, 1\right), \quad \mathbf{p} = (1-t)\mathbf{p}_i + t\mathbf{p}_j, \quad \mathbf{n} = \text{normalize}\big((1-t)\mathbf{n}_i + t\mathbf{n}_j\big). \tag{51}$$

The corner normals $\mathbf{n}_i$ are the analytic gradients of the trilinear field at the corner. The edge normal is the interpolation of the two corner normals, not the normalized edge direction. This is what makes the QEF preserve the smooth surface normal.

## 13.3 Accumulation

The normal equations are accumulated:

$$A = \sum_e \mathbf{n}_e \mathbf{n}_e^T + \epsilon I, \tag{52}$$

$$\mathbf{b} = \sum_e \mathbf{n}_e(\mathbf{n}_e \cdot \mathbf{p}_e) + \epsilon \bar{\mathbf{p}}. \tag{53}$$

where $\bar{\mathbf{p}}$ is the average crossing and $\epsilon = 10^{-2}$. The matrix is symmetric positive semidefinite; the shift makes it invertible. Cramer's rule is used because it is branch-free and matches the CPU's GLM evaluation order. A determinant below $10^{-6}$ returns zero and the caller falls back to the mass point.

## 13.4 Corner snap

If any corner SDF is exactly zero, the vertex is snapped to that corner:

$$\mathbf{x} = C_i. \tag{54}$$

Why? Neighbouring cells sharing the corner would otherwise solve slightly different QEF systems (their normals and crossings differ in the low bits) and their vertices would drift apart, splitting the shared edge into two one-sided edges. Snapping to the exact corner makes every incident cell produce the same bits. This is a classic dual-contouring robustness fix and it is essential for watertight terrain.

### 13.5 Sentinel handling

An edge with an infinite endpoint is skipped. If no usable edge remains but there are finite sign-changing edges blocked by a sentinel, the shader averages the finite crossings instead of returning the cell centre. The fallback keeps the vertex on the real surface segment rather than floating in the middle of the cell.

### 13.6 Clamping

The result is clamped to the cell:

$$\mathbf{x} = \text{clamp}(\mathbf{x}, \mathbf{c}, \mathbf{c} + L). \tag{55}$$

A QEF solution outside the cell would make the dual mesh non-manifold and could place a vertex inside a neighbouring solid. The clamp is the last line of defence after regularization.

### 13.7 Memoization

The shader memoizes up to eight QEF positions per invocation. A single edge segment queries the same cells several times (owner selection, dedup, rotation, fan). The memo is a linear scan over eight entries; it is value-identical to recomputing because the QEF is a pure function of the node's eight corners and cube. If the memo is full, the slot is chosen by `id mod 8`; a collision simply evicts an older entry.

### 13.8 The payload

The elected writer reconstructs the final vertex from the node id:

```
uint o = (pc.vertexBase + slot) * 4u;
poolVerts[o+0] = vec4(p, 1.0);
poolVerts[o+1] = vec4(1.0, 1.0, data);
poolVerts[o+2] = vec4(n, uintBitsToFloat(uint(brush)));
poolVerts[o+3] = vec4(uintBitsToFloat(uint(type)), h);
```

**Listing 11:** Vertex payload layout.

The vertex is 64 bytes:

| Word | Bytes |
|---|---|
| 0 | position.xyz (12) + color.x (4) |
| 1 | color.yz (8) + texCoord.xy (8) |
| 2 | normal.xyz (12) + brushIndex bits (4) |
| 3 | textureType bits (4) + hsv (12) |

The `textureType` is stored as integer bits, not as a float value. The vertex input attribute is `R32\_SINT`, and the ray-tracing path uses `floatBitsToInt`; writing a numeric float would decode to a garbage mapping mode and drop every non-triplanar vertex.

### 13.9 One dense slot per node

Every surface node in the flattened table gets a dense slot, including nodes that the index stream never references. This is intentional:

- it makes slot assignment a simple table order;

- it makes the vertex address space known from the host count, so the span is an exact upper bound for every index;
- it lets the geometry pass elect one writer per referenced slot with `atomicExchange` instead of running a separate compaction pass.

Only referenced slots are written, and the published span is trimmed to the highest referenced slot plus one. The remaining holes are bounded by the number of surface nodes in the neighbourhood and are reported by the per-chunk utilization profile.

# 14 The geometry pass: indices and lazy vertices

## 14.1 One dispatch, two outputs

The host sizes both spans before submission:

$$V_{\text{cap}} = \text{activeVertexCount}, \qquad I_{\text{cap}} = \text{cross-edge topology bound} \tag{56}$$

(see Section 8). The geometry dispatch then runs one invocation per compacted scan cell:

```
void main() {
    uint gi = gl_GlobalInvocationID.x;
    if (gi >= pc.cellCount) return;
    scanCell(cellIds[gi]);
    if (pc.diag != 0u) { /* publish findCell/reuse counters */ }
}
```

**Listing 12:** Geometry stage.

Each invocation appends its triangles straight into the chunk's index span and lazily writes the vertex payload of every referenced slot. There is no intermediate raw-key stream, no second work list, and no host round trip between indices and vertices.

## 14.2 Index values

Every index is the host-assigned slot of the emitting surface node:

$$\text{slot} = \text{nodeSlot}(\text{node}) = (\text{flags} \gg 1) - 1. \tag{57}$$

If the slot is negative, the host's slot assignment and the walk disagree. The shader increments `invalidSlotCount` and `droppedCount` and fails the chunk rather than writing slot 0. Writing 0 would silently connect the corner to an arbitrary vertex and produce a stretched triangle. This condition is not retryable: more capacity cannot fix a flattening disagreement.

## 14.3 Vertex reconstruction

The vertex is rebuilt from the node, not carried through an intermediate stream:

1. QEF position from the node's eight corners and cube;
2. analytic normal at that position;
3. brush, falling back to the chunk's `fromBrush` when the node's brush is the discard sentinel;
4. HSV tint, texture type, and mapping data.

The payload is a pure function of the node. The first invocation that references a slot wins the `atomicExchange` on `nodeWritten` and writes the payload; the others do nothing. This keeps the traffic at one 64-byte write per referenced node even though a node appears in many triangle corners.

### 14.4 The index base and alignment

The chunk's index span is bound with an offset. The offset must be aligned to the device's minimum storage-buffer offset alignment. Since the element size is four bytes and the alignment can be 256, the host aligns the descriptor offset down and passes `relIndexBase` = the number of index elements between the aligned offset and the first span element. The shader writes

$$\text{poolIdx}[\text{relIndexBase} + i] = \text{slot}. \tag{58}$$

The descriptor range is

$$(\text{relIndexBase} + I_{\text{cap}}) \cdot 4 \tag{59}$$

bytes, which includes the padding. Without the extra padding, the last elements of the span fall outside the bound range and are never written, producing stretched triangles from stale pool data.

### 14.5 The vertex base and alignment

The vertex span is handled the same way with a 64-byte stride. The offset is aligned down; `relVertexBase` accounts for the 0–3 element padding. The shader writes

$$\text{poolVerts}[(\text{relVertexBase} + s) \cdot 4 + k]. \tag{60}$$

The range is

$$(\text{relVertexBase} + V_{\text{cap}}) \cdot 64. \tag{61}$$

Because 64 is a multiple of 4 and the largest required alignment is 256, a span start can be misaligned by at most three vertices. The padding logic is exact and costs at most 192 bytes per chunk.

### 14.6 Overflow accounting and growth

The shader never writes outside the reserved spans. A triangle that does not fit still advances the index counter and updates the maximum referenced slot, so the host reads:

| Counter | Meaning |
|---|---|
| `indexCount` | indices the chunk would need (3 per triangle) |
| `vertexExtent` | highest referenced slot + 1 |
| `indexOverflow` | triangles rejected by the index span |
| `vertexOverflow` | triangles rejected by the vertex span |
| `invalidSlotCount` | corners without a host-assigned slot |

If an overflow is reported, the service frees this chunk's reservation and asks the provider for

$$I_{\text{next}} = \max(I_{\text{cap}}, \text{indexCount}), \qquad V_{\text{next}} = \max(V_{\text{cap}}, \text{vertexExtent}), \tag{62}$$

then retries the dispatch with the same input and a freshly cleared `nodeWritten` buffer. The retry is bounded (three attempts) and scoped to one chunk; there is no global rebuild or device wait. A fixed break-list overflow or an invalid slot is not retryable and fails the chunk.

### 14.7 Empty chunks

A chunk with no crossing scan cell, or with no surface node in the flattened table, has no possible triangle. The service reports success with zero geometry and does not ask the pool for a zero-sized span. The publisher settles the revision as empty, so a later edit that introduces a surface rebuilds it normally.

# 15 Host orchestration: one submission per chunk

## 15.1 The driver

`OctreeTopologyCompute::computeResident` is the only entry point. It is called by `SceneRenderer::buildChunkResident` on a scene worker thread. The method holds a mutex for the whole chunk, so two workers never share the staging buffers or the descriptor set. The normal path is one submission:

1. the host flattens the neighbourhood and derives the cross-edge capacities;
2. the provider reserves a vertex/index span in the packed pools;
3. the host uploads the node tables and records one geometry dispatch;
4. the host waits for the fence and reads the counters;
5. the host publishes the span, or grows it once and retries.

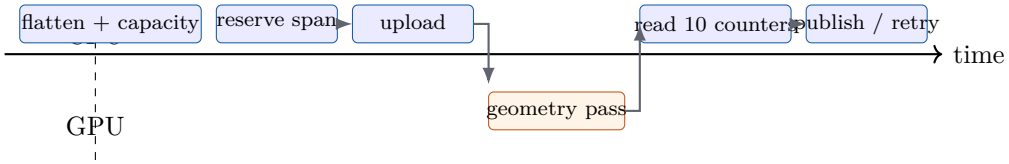



**Figure 12:** One chunk's normal timeline. The spans are reserved from the CPU cross-edge bound before submission; the single geometry pass writes the renderer's pools directly. The host reads integers after the fence.

## 15.2 The input upload

The input buffers are host-visible and coherent. `uploadInput` writes:

- 128 bytes per node for the node table;
- 4 bytes per cell for the scan list;
- ten counters (indices, extent, overflow, drops, diagnostics, and the distinct vertex write count);

and resets the counters. The upload is not a single `memcpy`: the node record is split across five arrays so the shader can use scalar layout and avoid a 128-byte struct with alignment surprises. The `nodeWritten` election buffer is device-local; it is cleared with `vkCmdFillBuffer` immediately before the dispatch, not uploaded from the host.

## 15.3 Capacity management

The input buffers grow to the largest chunk seen so far:

$$\text{nodeCap} \geq |\mathcal{N}|, \qquad \text{cellCap} \geq |\text{cells}|. \tag{63}$$

A grow recreates the buffers and bumps `bufferGeneration`. The descriptor writer checks the generation and rewrites bindings 0–9 only when it changes. In steady state, a chunk with the same capacity reuses the descriptors and pays zero descriptor writes. The device-local `nodeWritten` buffer follows `nodeCap`; it is not grown per chunk. The packed vertex/index spans belong to the renderer and are reserved per chunk.

## 15.4 Capacity plan and the provider callback

The provider is called before the dispatch with a demand derived from the CPU topology:

- vertex capacity = `activeVertexCount` (dense surface-node slot address space);
- index capacity = $\sum_c 6 \cdot \text{segments}(c) \cdot \text{crossEdges}(c)$;
- the cross-edge count and maximum segment span are carried for the profile.

The provider reserves a stable draw-entry slot and a variable-size span:

```
GpuMeshSpan span = ir.reserveGpuMeshSpan(chunkId,
    demand.suggestedVertexCapacity, demand.suggestedIndexCapacity,
    &cubeMin, &cubeMax, level, &cubeMin);
target.vertexBuffer = span.vertexBuffer;
target.baseVertex = span.baseVertex;
target.vertexCapacity = span.vertexCapacity;
target.indexBuffer = span.indexBuffer;
target.firstIndex = span.firstIndex;
target.indexCapacity = span.indexCapacity;
target.slotIndex = span.slotIndex;
```

**Listing 13:** Span reservation callback (shape).

If the provider cannot fit the requested capacities, the build fails without dispatching. The caller releases the reservation and leaves the revision uncommitted, so a later opportunity can retry.

### 15.5 Overflow growth

If the shader reports an index or vertex overflow, the service calls the release callback with the reservation it just failed to use. The renderer frees that unpublished span immediately (no draw command references it) and defers the free of any previously published span through the usual staged meta flush. The service then asks the provider for the exact reported capacities and retries the same chunk with a freshly cleared `nodeWritten` buffer. At most three attempts are made. Only this chunk's spans are released and re-reserved; other chunks and the global descriptor state are untouched. A fixed break-list overflow or an invalid slot fails immediately because capacity cannot fix it.

### 15.6 Publication

After the fence, the host reads the counters. If the chunk is valid:

1. `publishGpuMeshSpan(slot, vertexExtent, indexCount)` trims the reserved span to the referenced extent, frees the tail, and stages the draw command + bounds + LoD meta;
2. the scene publisher stores a `LoDMesh` with the slot, counts, base vertex, first index, bounds, brush, and mapping metadata;
3. the renderer's indirect cull sees the entry on the next frame.

The old span is released only after the new one is published. The release is staged for the cull frame after next, so no in-flight cull or draw can reference a freed range.

### 15.7 Why the host reads only counters

The GPU writes indices and vertices. The host reads only counters:

| Counter | Use |
|---|---|
| `indexCount` | indices appended; growth requirement |
| `indexOverflow` | triangles rejected by the index span |
| `vertexExtent` | published extent; growth requirement |
| `vertexOverflow` | triangles rejected by the vertex span |
| `vertexWriteCount` | distinct payloads written; utilization |
| `invalidSlotCount` | flattening disagreement (fatal) |
| `overflowCount` | fixed-capacity recursion failure (fatal) |
| `droppedCount` | total rejected triangles |
| `findCellCalls` | diagnostics only |
| `reuseHits` | diagnostics only |

No position, normal, or 64-byte vertex ever crosses the bus. This is the single most important property of the design: the host cannot accidentally stall on a vertex readback because there is no vertex readback API.

### 15.8 Error policy

A chunk build can fail in the following ways:

1. **flattening unavailable**: the chunk is skipped with a log;
2. **fixed-array overflow or invalid slot**: the chunk is failed and retried after the next edit; it is not partially drawn;
3. **span overflow**: the exact reported capacities are requested and the dispatch is retried once or twice, scoped to the chunk;
4. **pool span full**: release the slot and skip; the pool is sized for the worst case and this is a sizing bug, not a normal state.

The revision is committed only when the chunk is actually published or legitimately empty. A failed build is not recorded as up-to-date; a transient pool failure therefore retries instead of becoming a permanent hole.

## 16 Synchronization, queues, and lifetime

### 16.1 Queues in use

The engine has a main graphics queue and, when available, a distinct graphics-family queue for geometry/upload work. The topology compute pass is submitted on the graphics queue with a single-time command buffer and a fence, because its descriptor set and staging buffers are serialized by a global mutex. The geometry pass writes the renderer's packed pools; the cull, raster, shadow, and ray-tracing passes then read them on the same queue or after the documented cross-queue wait.

### 16.2 The barriers inside the topology submission

The submission starts by clearing the device-local election buffer and then barriers the fill against the compute stage:

```
vkCmdFillBuffer(cmd, nodeWritten, 0, nodeCount * 4, 0);
b.srcStageMask  = TRANSFER;
b.srcAccessMask = TRANSFER_WRITE;
b.dstStageMask  = COMPUTE_SHADER;
b.dstAccessMask = SHADER_READ | SHADER_WRITE;
```

**Listing 14:** Election-buffer clear barrier.

This is required because `atomicExchange(nodeWritten[node], 1)` elects one vertex writer per node; without the barrier the dispatch could read a stale flag from a previous chunk and skip a payload.

The submission ends with a compute→host barrier:

```
VkMemoryBarrier2 b{};
b.srcStageMask  = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT;
b.srcAccessMask = VK_ACCESS_2_SHADER_WRITE_BIT;
b.dstStageMask  = VK_PIPELINE_STAGE_2_HOST_BIT;
b.dstAccessMask = VK_ACCESS_2_HOST_READ_BIT;
vkCmdPipelineBarrier2(cmd, &dep);
```

**Listing 15:** Counter readback barrier.

This makes the integer counters visible to the host-mapped memory. The vertex/index writes are not read by the host, so they do not need a host barrier. The geometry pass is followed by a pool barrier:

```
b.srcStageMask  = COMPUTE_SHADER;
b.srcAccessMask = SHADER_WRITE;
b.dstStageMask  = VERTEX_INPUT | INDEX_INPUT | DRAW_INDIRECT |
                  COMPUTE_SHADER | ACCELERATION_STRUCTURE_BUILD;
b.dstAccessMask = VERTEX_ATTRIBUTE_READ | INDEX_READ |
                  SHADER_READ | ACCELERATION_STRUCTURE_READ;
```

**Listing 16:** Pool visibility barrier.

This is deliberately broad: the same pool range feeds the raster draw, the GPU cull, the shadow cascade, and the ray-tracing BLAS. The barrier runs once per chunk, not once per consumer.

### 16.3 The fence and the deferred release

The topology submission uses a fence. The host waits on the fence before reading the counters. An overflow retry releases the unpublished reservation immediately (no draw command can reference it) and clears the chunk's level data before the provider reserves again. A previously published span is released through the staged free queue; the actual free happens two cull frames later. This is necessary because the previous frame's cull or draw command buffers may still be in flight and still reference that span. The renderer does not wait for the whole device; it tracks the cull frame slot.

### 16.4 Consolidation

The vegetation instance buffers and the SDF flame anchors follow a similar pattern. A per-chunk placement dispatch writes into a scratch ring; the host reads the accepted instance count, allocates an exact persistent buffer, and copies the scratch slice on the GPU. The copy is submitted on the main thread after the placement fence, so no queue submission happens inside a deferred callback. The consolidated instance buffer is double-buffered; the active pair is read by the vegetation pass while the inactive pair is filled by the copy.

### 16.5 The queue-submission mutex

All Vulkan submissions go through a single global mutex keyed by `VkQueue`. This is intentionally coarse: the engine exposes several queue handles that can alias the same physical queue, and the microseconds saved by per-queue locking are not worth a submission race. The mutex covers:

- frame submit and present;
- `runSingleTimeCommands` and its async variant;
- upload-manager jobs;
- topology submissions;
- the vegetation exact-copy submission.

`vkDeviceWaitIdle` also takes the mutex. The earlier validation failure "`vkQueueSubmit2`: VkQueue is simultaneously used" came from calling `vkDeviceWaitIdle` without it while a worker was submitting. The fix is in `VulkanApp::deviceWaitIdle`, and cleanup now stops the CPU thread pools before the first idle wait.

### 16.6 Deferred destruction order

At shutdown the engine:

1. stops background threads (no new submissions);
2. waits for the device idle under the submission mutex;
3. drains pending command buffers;
4. runs deferred-destruction callbacks;
5. calls `clean()` on the app;
6. destroys tracked resources;
7. destroys the device.

The order matters because deferred callbacks capture renderer state. If the renderer were destroyed first, a callback that accesses `chunkBuffers` would read freed memory. The cleanup comment in `VulkanApp::cleanup` documents this exact hazard.

### 16.7 Why no host-side mesh wait

The host waits on the topology fence to read counters, not to read vertices. The wait is short (one dispatch), and it is per chunk on a worker thread, not on the render thread. The render thread only sees already-published draw entries. This is the difference between a GPU-resident pipeline and a GPU-assisted CPU pipeline: the host still synchronizes, but it synchronizes on integers, not on megabytes.

## 17 From pool span to rendered frame

### 17.1 Stable draw entries

The renderer is a *slotted* indirect renderer. Each chunk owns one stable draw-entry index; the entry is never compacted or renumbered while the chunk is alive. The packed pools are separate:

- a single large vertex pool (device-local, 64-byte elements);
- a single large index pool (device-local, 32-bit elements);
- a host-visible indirect-command array;
- a host-visible bounds/LoD-meta array.

A chunk's vertex and index data live in a variable-size span allocated by `PackedSpaceAllocator`. The draw entry stores the span's base vertex, first index, index count, and instance count.

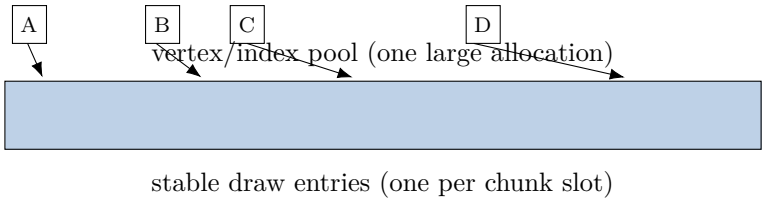



**Figure 13:** Packed pools and stable draw entries. Chunk spans are packed best-fit into one large vertex pool and one large index pool; the draw entry is a stable index into the indirect-command array. Rebuilding a chunk replaces only its span.

### 17.2 The draw command

A draw command is

$$\begin{aligned}\text{DrawIndexedIndirectCount}(&\text{indexCount}, \text{instanceCount} = 1,\\ &\text{firstIndex}, \text{vertexOffset} = \text{baseVertex},\\ &\text{firstInstance} = \text{slot}).\end{aligned} \tag{64}$$

The vertex shader uses `gl_InstanceIndex` only for the per-chunk vegetation/impostor path; the solid terrain vertex is a direct vertex attribute. The index stream is chunk-local: every index is a slot number in the chunk's vertex span, and `vertexOffset` rebases it at draw time.

### 17.3 Depth and offscreen composition

Vegetation is rendered to its own color and depth targets on a parallel async command buffer; the solid pass has a separate depth target. The composite pass tests the vegetation depth against the solid scene depth. This decoupling is why the vegetation pass can use a different render scale and why it can be skipped on steady-empty frames. The solid chunks themselves are drawn in the main solid pass, with a depth prepass and a shadow cascade.

### 17.4 Shadows

The shadow renderer reuses the same slots and the same hierarchy. A cascade cull emits one compacted command per chunk per cascade, reusing the already-selected LoD stamp from the main cull. The shadow vertex shader reads the same 64-byte vertex and index pool. No second mesh exists. The cascade copy lists and descriptor-buffer mirror include the brush map and wind field so the shadow tessellation stage can displace and classify the surface without a separate descriptor.

### 17.5 Ray tracing

The hybrid ray tracer builds a BLAS over the merged vertex/index pools. A chunk's geometry span is referenced directly:

- vertex buffer = the pool, vertex stride 64;
- index buffer = the pool, first index = span's first index;
- geometry flags = opaque or alpha-tested, depending on the material.

Because the span is published only after the geometry fence, the BLAS build never sees partially written vertices. A chunk rebuild requests a BLAS refresh; the proxy boxes are used for the water reflection path and for the fallback when the BLAS is not ready.

### 17.6 Material and mapping

The vertex carries brush, HSV, texture type, and mapping data. The solid fragment shader selects the material texture from the brush and the triplanar/projection mapping from the texture type. Because the CPU never computed a vertex, the mapping data must come from the node: the shape leaf paints it, internal nodes inherit the dominant child, and the geometry pass writes it into the vertex. A mapping bug therefore shows up as a texture scale collapse, not as a geometry error.

### 17.7 The frame graph

The main frame is roughly:

1. frame fences and pending command-buffer processing;
2. uploads and deferred destroys;
3. GPU cull for solid, water, SDF, bbox, vegetation;
4. depth prepass;
5. shadow cascades;
6. solid color pass;
7. water, vegetation, video, SDF passes;
8. post-process composition;
9. ImGui and present.

The topology pass is not in this list: it runs on worker threads when a chunk is built, not per frame. A frame only consumes published spans.

## 18 GPU culling and the hole-free LoD gate

### 18.1 One dispatch, many streams

The solid `IndirectRenderer` owns a single compute dispatch (`IndirectRenderer.comp`) that culls:

- solid terrain chunks;
- SDF debug cubes;
- mesh bounding boxes;
- main-view vegetation (blade tufts and impostors);
- three shadow cascades for solid and vegetation.

The different streams are appended in the same dispatch and written to separate outputs. This avoids one dispatch per stream and keeps the per-frame CPU cost constant.

### 18.2 Frustum test

The view-projection matrix is a push constant. The shader extracts six normalized planes once per workgroup and tests an AABB by its most positive vertex. The test is conservative: it can keep a box that is just outside, but it never rejects a box that intersects the frustum.

### 18.3 The hierarchical rung gate

The LoD gate is the most delicate part of the renderer. Each published chunk is a cell of a dyadic ladder. The goal is to draw exactly one rung per region, with no holes and no overlaps. A naive distance-to-centre band fails because the centres of parent and child are not nested at the same position; it can reject every rung of a column.

The engine uses the per-entry form of the classic CDLOD recursion. For an entry at level $r$ with cell size $L$:

$$L_{\text{base}} = L/2^r, \tag{65}$$

$$R_{\text{self}} = L_{\text{base}} \cdot \text{bias} \cdot r, \tag{66}$$

$$\text{recurseSelf} = (r > 0) \wedge \text{dist}_{\text{AABB}}(\mathbf{c}, L) < R_{\text{self}}. \tag{67}$$

The parent cell is the aligned double-size cube containing the entry. The parent range is $L_{\text{base}} \cdot \text{bias} \cdot (r+1)$. The entry is selected when

$$\text{recurseParent} \wedge \neg\text{recurseSelf}, \tag{68}$$

with the coarsest rung selected when it does not recurse, and the finest rung selected whenever its parent recursed (there is nothing finer to clamp to). Both distance and range are monotone along the chain, so exactly one level satisfies the transition for every camera position.

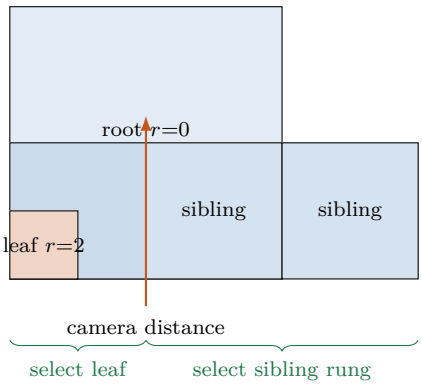



**Figure 14:** The hierarchical gate selects one rung per region. The leaf is selected near the camera; the sibling rung is selected farther away. A region can never lose all rungs because the parent/child ranges are monotone along the chain.

## 18.4 The visible-LoD stamp

The gate's decision is written into `visibleLods[s]` as

$$\text{stamp} = \begin{cases} r, & \text{kept}, \\ \texttt{0xFFFFFFFE}, & \text{rejected LoD entry}, \\ \texttt{0xFFFFFFFF}, & \text{no LoD data (legacy)}. \end{cases} \tag{69}$$

The shadow cascade cull reads this stamp instead of re-deriving the selection with a stale camera position. This fixes the old bug where far chunks cast finest-mesh shadows for a mesh the main pass never drew.

## 18.5 Vegetation follows the chunk

Vegetation is not a separate culling stream. The per-solid-entry vegetation table holds the instance count and first instance for the chunk. When the solid entry survives the frustum and rung gate, the same dispatch emits a blade-tuft command and an impostor command for the chunk. The vegetation bounding box is the chunk AABB, so it is culled exactly with the terrain. A chunk with no vegetation has `instanceCount=0` and is skipped.

### 18.6 Water pin

The water stream uses the same gate but pins the finest rung ($r = 0$) at every distance. Small floating water volumes (mirror spheres) are sub-cell features past rung 2–3; the coarse rungs mesh to nothing and would leave a hole. There are few water draws, so finest-always is affordable. The shadow, backface, and RT passes follow the same stamp.

### 18.7 Indirect count

The cull writes a compacted command array and a count. The draw uses the indirect-count command with the count buffer. The count is read back asynchronously for the stats overlay; the draw itself never waits on the CPU. The count buffer is reset by `vkCmdFillBuffer` before the dispatch.

## 19 GPU-resident effects: grass and fire

### 19.1 The shared span

Vegetation and fire both need information from the chunk's surface, but neither needs a CPU copy of the triangle mesh. They consume the same published span:

- vegetation reads the vertex and index pools directly;
- fire reads the node SDFs on the host build and emits anchors.

Both are keyed by chunk id and both participate in the chunk's lifecycle: a republish replaces the old data, a delete tombstones it, and a rebuild creates a new version.

### 19.2 Vegetation placement

The placement shader `VegetationPlacement.comp` reads the chunk's vertex span, treats every triangle as a potential grass emitter, and:

1. rejects triangles with no grass-brush vertex;
2. computes the triangle area;
3. expands a stochastic number of virtual slots proportional to area;
4. samples a barycentric point;
5. applies the dominant-vertex brush test;
6. applies the biome noise gate;
7. selects a blade variant and a rotation fraction by position hash;
8. writes a 32-byte instance record.

The placement output goes into a scratch ring. The host reads the accepted count, allocates an exact persistent buffer, copies the scratch slice, and publishes the chunk. No instance byte reaches the host. A coarse rung with thousands of tufts is capped at `kPlacementScratchPerChunk` to bound the far-field draw; the cap is a deliberate density reduction, not a silent overflow.

### 19.3 Why not one instance per triangle?

The old CPU code expanded slots on the CPU, one per area-weighted sample. The GPU port does the same expansion in the shader: each invocation is one triangle and loops over its stochastic slot count. This avoids a prefix-sum pass and a second dispatch. The trade-off is that the output order is non-deterministic; the instance record itself is deterministic because every random draw is seeded by the chunk, triangle, and slot.

### 19.4 The vegetation draw

The cull emits a blade-tuft command with `indexCount=36` (12 blades, one triangle each) and an impostor command with `indexCount=6`. The instance count and first instance come from the per-chunk table. The vegetation vertex shader reads the instance record:

- position,
- normal,
- variant and rotation,
- baked height,

and expands the procedural blade geometry. The impostor path samples the captured blade atlas and reconstructs depth from the captured depth array.

### 19.5 Lava/fire anchors

The SDF fire effect is driven by brush-4 surfaces. The old collector sampled the CPU mesh; after the GPU-only change it received an empty geometry. The replacement resolves anchors during the chunk build:

1. for every brush-4 surface node that owns a vertex slot, evaluate the node's trilinear SDF and gradient at its centre;
2. take one Newton step to the zero crossing;
3. clamp to the node cube;
4. emit a position, normal, and seed.

The anchors are carried on `LoDMesh` to the main thread and ingested by `SdfRenderer`. The renderer caps the number per chunk, deduplicates nested LoD rungs on an 8 m grid, builds a flame instance list, and stages the SDF scene. The volumetric fire path then renders the flames through the same generic SDF scene as smoke and bullets.

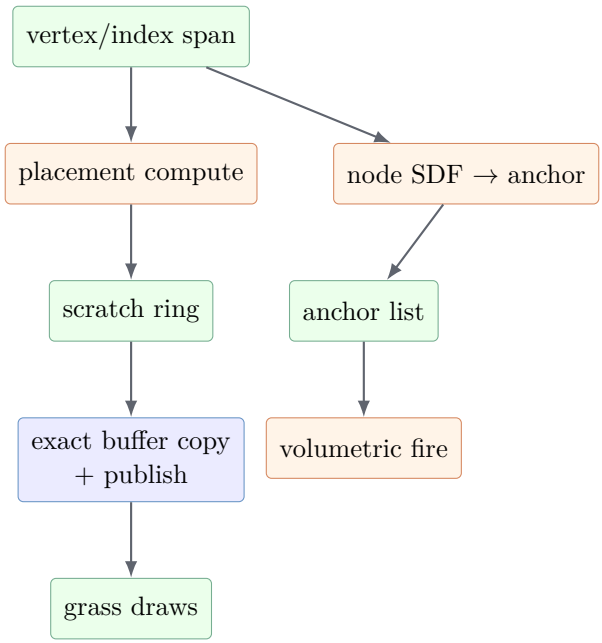



**Figure 15:** Two GPU-resident effects sharing the chunk span. Vegetation writes instances and copies them to an exact persistent buffer; fire resolves anchors during the build and hands them to the SDF renderer.

### 19.6 Ownership and cleanup

Both effects register with the chunk lifecycle:

- a republish replaces the chunk's instance buffer/anchor entry;
- an empty chunk tombstones the old entry;
- a chunk delete removes the entry and defers the buffer destroy until all pending submissions complete;
- app shutdown stops the worker pools before the device idle wait.

The effect data is therefore never left pointing at a freed span.

## 20 Performance and memory

### 20.1 Cost model

The per-chunk cost is

$$C_{\text{chunk}} = C_{\text{flatten}} + C_{\text{capacity}} + C_{\text{reserve}} + C_{\text{upload}} + C_{\text{dispatch}} + C_{\text{read}} + C_{\text{publish}}. \tag{70}$$

Flattening and capacity accounting scale with the number of nodes in the neighbourhood; the dispatch scales with the number of scan cells and active edges. The host reads ten integers after one fence; there is no counting submission between them. The one retryable cost is a defensive span overflow, which reuses the uploaded input and only re-reserves the chunk.

### 20.2 Measured topology rates

The debug profiler in `OctreeTopologyCompute` reports cumulative averages every 256 chunks when `VULKAN_SETUP_DIAG=1`. The following windows were captured from the earlier two-stage path; they are retained as a baseline because the single-pass path removes the first submission and the intermediate raw-key traffic, so it cannot be slower for the same inputs.

| Chunks | avg ms/chunk | avg cells | avg nodes |
|---|---|---|---|
| 256 | 8.3 | 1031 | 3761 |
| 512 | 9.3 | 911 | 3440 |
| 768 | 10.0 | 885 | 3382 |
| 1280 | 18.9 | 788 | 3053 |

The later window is slower despite fewer cells and nodes because the workers contend with the render thread and with descriptor/buffer growth. The uncontended floor reported for the last window of an earlier capture was about 4.2 ms/chunk. The difference is a convoy effect, not a super-linear algorithm.

### 20.3 Throughput

The geometry pass is one invocation per scan cell. For a chunk with about 900 scan cells and 3.4k nodes, the baseline 8–19 ms includes the host-side flatten and upload, two submissions, and the fence wait. The current path removes the counting submission; its remaining cost is dominated by the host-side flattening plus the adaptive per-cell branch divergence and break collection, not by the QEF solve or the index append.

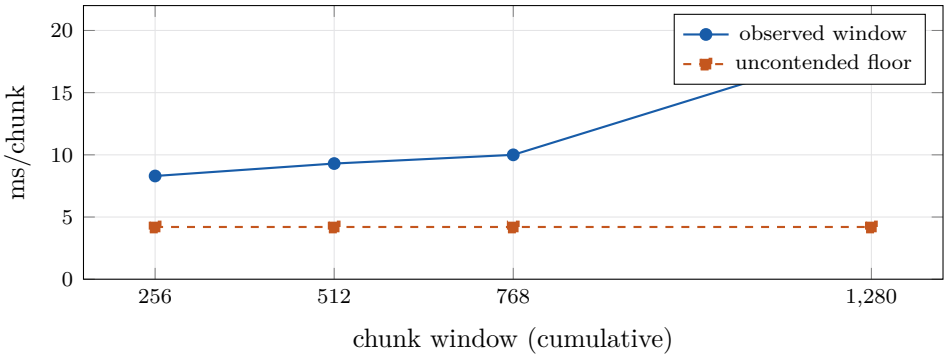



**Figure 16:** Observed topology cost per chunk. The floor is the cost when the worker pool is not contending with the render thread or with buffer growth.

### 20.4 Per-chunk utilization profiling

Every build records a `ChunkMeshProfile`: chunk id, layer, LoD, cross edges, the maximum per-edge segment bound, reserved vertex/index capacities, distinct payloads actually written, the published vertex extent, appended indices, overflow counts, dispatch attempts, and whether the span grew. Because the counters are the ones the build already reads back, profiling adds no GPU readback and no extra traversal. The registry keeps the latest profile per `(layer, chunk)` and caches a sorted snapshot, so the overlay pays no sort unless a build changed the set.

The overlay reports aggregate totals plus per-chunk minimum, mean, median, and maximum for vertex utilization, index utilization, and the byte-weighted combined utilization

$$U_{\text{combined}} = \frac{64\,N_v + 4\,N_i}{64\,C_v + 4\,C_i}, \tag{71}$$

where $N$ is the actual count and $C$ the reserved capacity. It also counts chunks below 25 % combined utilization and chunks with an overflow or growth event. The I/O boundary calls `logMemoryUtilization()`, which prints the aggregate; setting `VULKAN\_CHUNK\_PROFILE` additionally prints one line per chunk from that path and from the periodic stream diagnostic.

### 20.5 Memory budget

The reference configuration reserves:

| Resource | Size | Notes |
|---|---:|---:|
| Solid vertex pool | 768 MB | 64 B × 12M elements |
| Solid index pool | 192 MB | 4 B × 48M elements |
| Water vertex pool | 96 MB | 64 B × 1.5M elements |
| Water index pool | 96 MB | 4 B × 24M elements |
| Vegetation concat A/B | 256 MB × 2 | 8M instances × 32 B |
| Vegetation baked A/B | 32 MB × 2 | one float per instance |
| Vegetation scratch | 80 MB | 2 × 10 × 131k × 32 B |
| Texture arrays | 720 / 747 MB | five maps, 28 layers |

The pools are preallocated once. A publish only carves or frees a span; it never allocates a Vulkan buffer. This is why the steady-state path has zero `vmaCreateBuffer` calls and why a streaming burst cannot fragment the allocator.

### 20.6 Streaming behaviour

Chunks are drained at a controlled rate (16 per frame in the scene publisher, 10 per frame for vegetation). A full drain would submit hundreds of topology builds at once; on a 4 GB iGPU that trips the amdgpu watchdog. The rate limit keeps the submit queue bounded and spreads the GPU work over frames. The vegetation and fire effects lag the solid publish by one or two frames; that is acceptable because the chunk is not visible until its draw entry is published.

### 20.7 Failure costs

The expensive failure is a span-overflow retry: it releases the chunk's reservation and re-runs the geometry dispatch with the exact reported capacities. The cross-edge bound is a topology bound, so this should be rare; it is bounded to three attempts. A fixed break-list overflow or an invalid slot fails the chunk instead of retrying. A pool exhaustion is a sizing bug: the pools are preallocated for the worst measured scene, and a new scene must bump the budget deliberately.

### 20.8 Interpretation

The pipeline is not "free". It pays:

- a per-chunk flatten and cross-edge capacity pass;
- one upload of the node neighbourhood;
- one submission and one fence wait;
- a small host read of ten counters;
- a per-chunk span reservation and descriptor update.

It saves:

- a GPU→CPU vertex readback;
- a CPU-side QEF solve for every vertex;
- a host-side triangle append and a staging upload;
- a CPU-to-GPU copy of the index stream.

On a discrete GPU the readback and staging are the dominant costs; on an integrated GPU they are even more expensive because the memory is shared. The measured topology cost is the price of the split, and it is paid on worker threads, not on the render thread.

## 21 Correctness invariants and validation

### 21.1 The invariants

The following invariants must hold after any change to the pipeline.

**Invariant 1** (Single Cartesian producer)**.** *For a runtime chunk, every Cartesian vertex is written by the single* `OctreeTriangles.comp` *geometry pass. No CPU code writes a chunk vertex. The offline exporter is the only exception and is explicitly marked.*

**Invariant 2** (Slot determinism)**.** *The host assigns vertex slots in node-table order. A rebuild with the same octree revision produces the same slot for the same surface node.*

**Invariant 3** (Span lifetime)**.** *A draw entry references a span only after the geometry fence has signaled and the overflow counters are zero. The old span is not freed until the cull frame that could still reference it has retired; an unpublished reservation released during an overflow retry has no draw command and may be freed immediately.*

**Invariant 4** (Index locality)**.** *Every index written by the geometry pass is a chunk-local slot* $s \in [0, \mathit{vertexCapacity})$ *and is rebased by* `vertexOffset=baseVertex` *at draw time.*

**Invariant 5** (Owner consistency)**.** *For a shared edge, both neighbouring chunks select the same owner, sample the same four quadrants, and emit the same sub-segment list. The owner key is global (size, min, node index).*

**Invariant 6** (Winding oracle)**.** *Triangle winding is derived from the producing cell's corner signs, not from a QEF normal. A degenerate owner interpolation falls back to the producing cell's signs.*

**Invariant 7** (No silent truncation)**.** *A break-list, stack, or span overflow sets a counter. A span overflow is reported with the exact requirement and retried under a bounded, per-chunk growth; a fatal overflow fails the chunk. No path publishes a partial span, and no path writes outside the reserved spans.*

### 21.2 Validation layers

Development builds enable Khronos validation, synchronization validation, best-practices validation, and, when requested, GPU-assisted validation. The engine treats every validation message as a bug. The most relevant checks for this pipeline are:

- descriptor range and offset alignment;
- storage-buffer access outside the bound range;
- synchronization hazards between compute writes and vertex/index reads;
- queue submission threading;
- image layout transitions in the offscreen vegetation/SDF targets.

### 21.3 GPU-assisted validation

GPU-assisted validation instruments shader resource access. It is especially useful here because a wrong `relVertexBase` or a stale span may not crash: it reads another chunk's vertex and renders a stretched triangle. The GPU-assisted run catches the out-of-range descriptor access at draw time. The pass is slow and is used as a targeted check, not as the default.

### 21.4 Self-check against the CPU walker

A diagnostic mode can rebuild a chunk with the CPU walker and compare the triangle sets. The comparison is quantization-tolerant because the GPU and CPU may differ in the last bits of a QEF solve. The check is:

- same triangle count;
- same set of quantized triangle centroids;
- same material assignment;
- vertex deviation below a tolerance.

A large deviation indicates a flattening error (wrong cell, wrong LoD, wrong owner), not a floating-point difference.

### 21.5 Determinism tests

A chunk is built twice from the same revision. The vertex spans must be byte-identical and the index stream must be a permutation of the first. The triangle set is compared order-insensitively. This catches:

- an unstable table order;
- a slot assignment that depends on a hash map iteration;
- an atomic append that changes the triangle set (it should not).

### 21.6 Synchronization validation

The topology pass is checked for:

- transfer→compute visibility of the election-buffer clear;
- compute→host visibility before the counter read;
- compute→vertex/index visibility before the draw;
- no read of the pool while the geometry dispatch is still writing;
- no free of a published span still referenced by an in-flight cull.

The vegetation copy is checked for the same pattern: placement fence → host read → copy submission → publish → consolidation copy.

### 21.7 The queue-threading bug

A release run produced

```
vkQueueSubmit2():  THREADING ERROR : object of type VkQueue is
simultaneously used...
```

The cause was `vkDeviceWaitIdle()` running without the submission mutex while a worker submitted a topology or upload command. The fix:

- `deviceWaitIdle()` takes the global submit mutex;
- cleanup stops the CPU pools before the first idle wait.

After the fix, a full scene load and shutdown produced no threading or validation messages. This is an example of why the synchronization section is part of the algorithm, not an afterthought.

### 21.8 Regression checklist

Any change to the mesher should run:

1. a debug build;
2. a load of the reference scene with all effects;
3. a validation scan of `logs/run.log`;
4. a GPU-assisted run on a small scene;
5. a CPU-vs-GPU self-check on a chunk with a LoD boundary;
6. a rebuild/delete/rebuild cycle on the same chunk id.

If any of these fails, the change is not ready.

## 22 Discussion, limitations, and future work

### 22.1 What the split buys

The CPU octree is a semantic store: it knows the SDF, the brush, the mapping, the version, and the LoD ladder. The GPU mesher is a pure function from a flattened neighbourhood to a dual mesh. This separation makes both sides simpler:

- the CPU never needs a triangle mesh to answer “what is the material at this cell?”;

- the GPU never needs a pointer, an allocator, or a tree lock;
- the renderer never needs a conversion pass;
- the exporter can reuse the CPU walker as a reference.

### 22.2 The remaining host cost

The host still pays for the flattened upload and the two fence waits per chunk. The measured 8–19 ms/chunk is not negligible. The cost is on worker threads, so it does not directly stall the render thread, but it consumes CPU cores and memory bandwidth. The next step would be to batch several chunks into one submission and share one descriptor set, as the perf reports propose. That would amortize the submit/fence cost and the descriptor writes. The current per-chunk design was chosen for incremental streaming and easy failure isolation; batching is a throughput optimization, not a correctness requirement.

### 22.3 The mesh is not the only consumer

The same span feeds:

- the raster solid pass;
- the shadow cascades;
- the ray-tracing BLAS;
- the vegetation placement compute;
- the SDF fire anchors (via node SDFs).

This is why the span protocol is centralized in `IndirectRenderer`. If each consumer allocated its own copy, the memory and the synchronization surface would explode. The packed pool is the single source of geometry truth for the frame.

### 22.4 Adaptive LoD and cracks

The adaptive stitching is exact for the chunk-local walk, but it relies on both chunks seeing the same four quadrant cells. The one-cell margin guarantees this for a neighbour that is at most one cell larger. A pathological tree with a much larger neighbour (more than one cell) could still sample outside the margin. The flattening uses the largest target cell in the subtree, not the chunk's own cell, precisely to cover this case. A future change that shrinks the margin must re-run the CPU-vs-GPU self-check on a chunk with a coarse neighbour.

### 22.5 The coarse-rung density cap

Vegetation placement caps a single coarse rung at a fixed instance count. This is a deliberate visual/performance trade: a root-scale chunk can request hundreds of thousands of tufts, which is both expensive and visually indistinguishable from a smaller count at its distance. The cap is logged, not silent. A future distance-density model could replace the hard cap with a screen-space or distance-based thinning curve.

### 22.6 The fire anchors

The fire effect now derives anchors from the node SDFs rather than from the triangulated surface. This is cheaper and fits the GPU-only mesh, but it is coarser than the old triangle sampling. The Newton step lands on the trilinear zero crossing; for a primitive sphere or cylinder this is accurate. For a highly non-planar cell, the anchor can be a few centimetres off the true QEF surface. The

flame scale is large enough that this is not visible. A future improvement could run a small GPU placement pass over the brush-4 triangles and read back only the anchor records.

### 22.7 What would break the design

The following changes would violate the contract and require a redesign:

- reading a vertex back to the host (destroys the no-readback property);
- letting the CPU write a chunk vertex (destroys the single-producer invariant);
- freeing a span without a cull-frame delay (use-after-free);
- changing the node table order without re-running the exporter comparison (changes slot assignment and the BLAS);
- dropping a triangle on overflow instead of failing (silent holes).

### 22.8 Future work

Three directions are natural:

1. **Batched topology.** Flatten all chunks of a layer into one arena, record one geometry dispatch for the batch, and read all counters with one map. This is the highest-leverage performance change.
2. **GPU-side fire placement.** A compute pass over brush-4 triangles could produce anchors in a device buffer and the SDF scene could consume them directly, avoiding the host anchor list.
3. **Compressed vertex payload.** The 64-byte vertex is generous; a packed normal/brush/mapping format could halve the pool and the bandwidth, at the cost of a decode in the vertex shader and the BLAS build.

## 23 Conclusion

This report followed one geometric path from an author's implicit shape to a shaded pixel. The CPU evaluates the SDF, builds an adaptive octree of corner samples, simplifies it into a LoD ladder, and flattens a chunk's neighbourhood into compact tables. The GPU walks the twelve cube edges, stitches LoD boundaries, solves one dual vertex per surface cell, and writes the final 64-byte vertices and chunk-local indices directly into the renderer's persistent pools. The host sees counters, slots, and versions—not vertices.

The key ideas are:

- **One Cartesian producer.** The geometry pass is the only runtime code that writes a vertex. This eliminates an entire class of CPU/GPU divergence and makes the offline exporter a meaningful reference.
- **Root-consistent flattening.** The GPU table is a serialisable snapshot of the exact neighbourhood the CPU walker would traverse, including the one-cell margin for coarse neighbours.
- **Break-based stitching.** The edge is split at every cell boundary, not at a fixed table of configurations. Two chunks resolve the same edge to the same owner and the same sub-segments.
- **Span publishing.** A chunk's geometry is a reserved range in a large pool, published only after the geometry fence and freed only after the cull frames that could reference it have retired.
- **One hierarchy for everything.** Raster, shadows, ray tracing, vegetation, and fire all consume the same published span or the same node metadata.

The result is not a toy mesher. It streams an adaptive, crack-free surface from a mutable SDF octree into a real-time renderer without a vertex readback, and it keeps the host's work proportional to the

number of chunks and nodes, not to the number of triangles. The measured costs and the invariants in this report are the contract a future change must preserve.

# A Storage layouts and constants

## A.1 Octree node record (host)

| Field | Type | Bytes |
|---|---|---|
| `sdf[8]` | float | 32 |
| `brushIndex` | int | 4 |
| `textureType` | int | 4 |
| `data` | vec2 | 8 |
| `hsv` | vec3 | 12 |
| `blockId` | uint | 4 |
| `bits` | uint8 | 1 |
| `lod` | uint8 | 1 |
| `chunkLod` | uint8 | 1 |
| `version` | uint | 4 |

## A.2 GPU node table (flattened)

| Binding | Record | Bytes |
|---|---|---|
| 0 | `vec4 sdf0` | 16 |
| 1 | `vec4 sdf1` | 16 |
| 2 | `vec4 cubeMinSize` | 16 |
| 3 | `ivec4 meta` | 16 |
| 4 | `vec4 nodeHsv[2]` | 32 |
| 5 | `ivec4 child03` | 16 |
| 6 | `ivec4 child47` | 16 |
| total | | 128 |

## A.3 Packed vertex record

| Word | Field | Bytes |
|---|---|---|
| 0 | position.xyz + color.x | 16 |
| 1 | color.yz + texCoord.xy | 16 |
| 2 | normal.xyz + brush bits | 16 |
| 3 | textureType bits + hsv | 16 |
| total | | 64 |

### A.4 Push constants

| Field | Meaning |
|---|---|
| `cellCount` | scan cells in this dispatch |
| `targetLod` | rung to scan |
| `fromId` | descent anchor |
| `fromBrush` | fallback brush |
| `rootMinSize` | world root $(x, y, z, L)$ |
| `worldSize` | root edge for epsilon |
| `vertexBase` | span-relative vertex base |
| `vertexCapacity` | vertex span bound |
| `indexBase` | span-relative index base |
| `indexCapacity` | index span bound |
| `nodeCount` | node-table entries (election-buffer bound) |
| `diag` | diagnostic counters on/off |

### A.5 Constants

| Name | Value | Purpose |
|---|---|---|
| `kTrianglesPerSegment` | 2 | index bound per segment |
| `kIndicesPerTriangle` | 3 | index bound per triangle |
| `kMaxSegmentsPerCrossEdge` | 64 | break-list bound |
| `kMaxSpanRetries` | 3 | overflow-growth attempts |
| `BREAK\_CAP` | 64 | final breaks per edge |
| `LOCAL\_BREAK\_CAP` | 32 | local breaks per interval |
| `TASK\_STACK` | 256 | interval stack |
| `POS\_MEMO` | 8 | QEF memo |
| `kInstanceStride` | 32 | vegetation instance |
| `kMaxVegChunks` | 4096 | vegetation chunk cap |
| `kPlacementScratchPerChunk` | 131072 | vegetation coarse cap |

# B The complete scan-cell algorithm

**Algorithm 1** GPU scan cell (one invocation per cell)

```
cell ← cell descriptor from table
corners[8] ← node SDF samples
for e = 0 to 11 do
    (i, j) ← EDGES[e]
    if sign(corners[i]) = sign(corners[j]) then
        continue
    end if
    span ← makeEdgeSpan(e, cell)
    if span.end − span.start ≤ 2ϵ then
        continue
    end if
    breaks ← {span.start, span.end}
    collectBreaks(span, span.start, span.end)
    if overflow then
        fail chunk
    end if
    sort/unique(breaks)
    for k = 1 to |breaks| − 1 do
        if breaks[k] − breaks[k − 1] > 2ϵ then
            emitSegment(span, breaks[k − 1], breaks[k], sign_i, sign_j)
        end if
    end for
end for
```

# C Glossary

**Chunk**
A cube of edge $\Delta$ whose subtree is flattened and meshed as one unit. A chunk root is published as a draw entry.

**Rung**
A level of the chunk ladder. $r = 0$ is the finest published level; larger $r$ are coarser ancestors.

**Scan cell**
A cube at the target rung whose twelve edges are scanned for sign changes.

**Edge span**
The interval of an active cube edge, with its fixed coordinates and epsilon.

**Break**
A coordinate inside an edge span where a neighbouring cell boundary forces a split.

**Owner**
The surface cell that determines the winding and leads the ring of a sub-segment.

**Cross edge**
A scanned cube edge whose two stored corner samples have opposite signs. The per-chunk count sizes the index span.

**Slot**
The dense vertex index assigned to a surface node by the host; the geometry pass writes the

payload once per referenced slot.

**Span**
A reserved range in the packed vertex/index pools.

**Stamp**
The per-entry LoD keep/reject decision written by the cull and consumed by the shadow/RT paths.

## References

[1] W. E. Lorensen and H. E. Cline. Marching cubes: A high resolution 3D surface construction algorithm. *ACM SIGGRAPH Computer Graphics*, 21(4):163–169, 1987.

[2] T. Ju, F. Losasso, S. Schaefer, and J. Warren. Dual contouring of Hermite data. *ACM Transactions on Graphics (SIGGRAPH)*, 21(3):339–346, 2002.

[3] S. F. F. Gibson. Constrained elastic surface nets: Generating smooth surfaces from binary segmented data. In *Proc. MICCAI*, pages 888–898, 1998.

[4] J. C. Hart. Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces. *The Visual Computer*, 12(10):527–545, 1996.

[5] I. Quilez. Smooth minimum. https://iquilezles.org/articles/smin/, 2013.

[6] S. Strugar. Continuous distance-dependent level of detail for rendering heightmaps. *Journal of Graphics, GPU, and Game Tools*, 14(4):57–74, 2009.

[7] Khronos Group. *Vulkan 1.4 Specification*. https://www.khronos.org/registry/vulkan/, 2024.

[8] C. Ericson. *Real-Time Collision Detection*. Morgan Kaufmann, 2004.

[9] G-Truc Creation. *OpenGL Mathematics (GLM)*. https://glm.g-truc.net/, 2024.

[10] J. C. Hart, D. Sandberg, and S. M. Omohundro. Ray tracing implicit surfaces. In *SIGGRAPH Course Notes*, 1993.